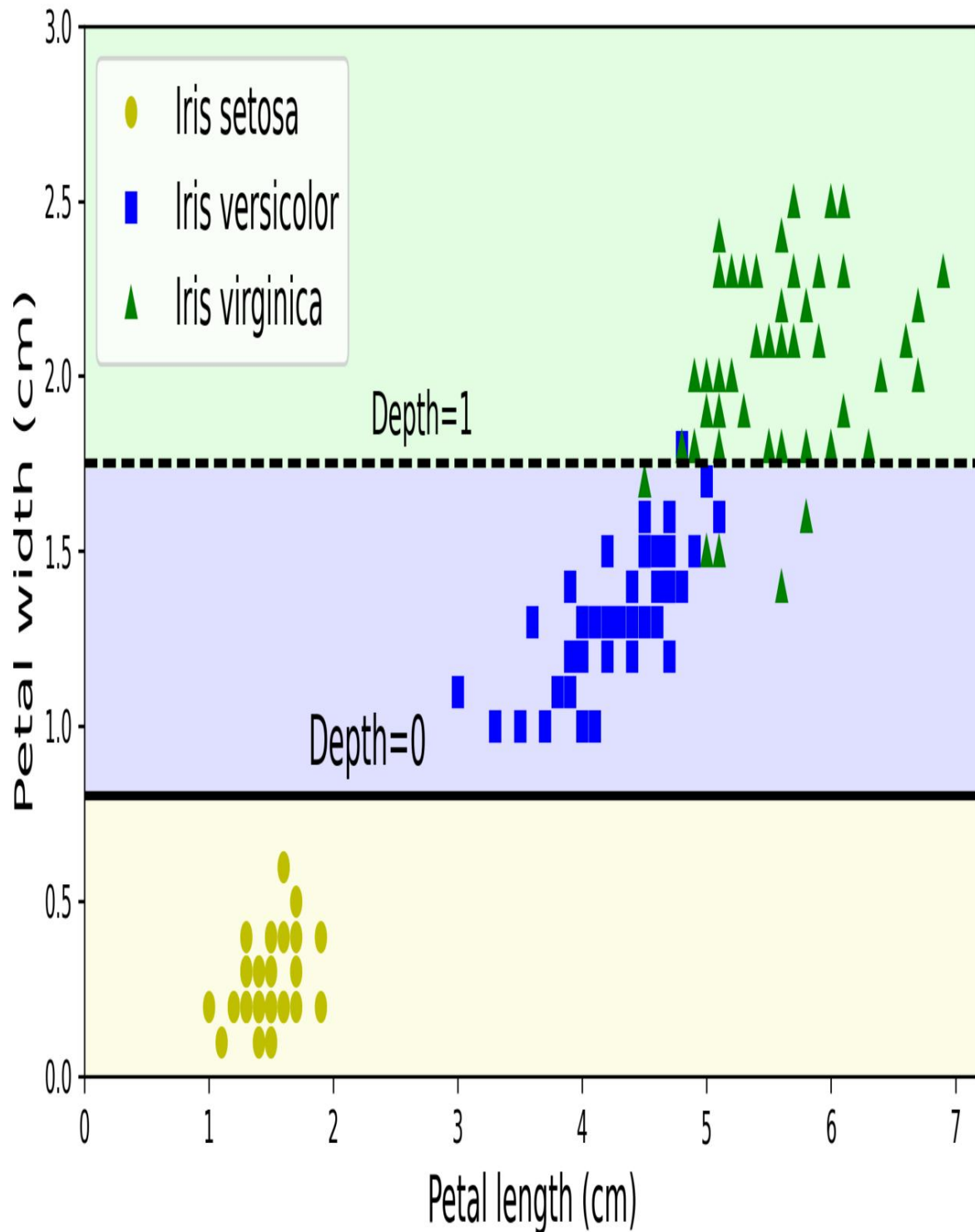




Hình 6-9. Việc huấn luyện lại cùng một mô hình trên cùng một tập dữ liệu có thể tạo ra một mô hình rất khác.

May mắn thay, bằng cách lấy trung bình các dự đoán từ nhiều cây quyết định, ta có thể giảm đáng kể phương sai. Tập hợp các cây quyết định như vậy được gọi là Rừng ngẫu nhiên (Random Forest), và đây là một trong những loại mô hình mạnh mẽ nhất hiện nay, như chúng ta sẽ thấy trong chương tiếp theo.

Bài tập

1. Độ sâu xấp xỉ của cây quyết định được huấn luyện (không có ràng buộc) trên tập dữ liệu huấn luyện gồm một triệu mẫu là bao nhiêu?
2. Độ tập Gini của một nút nói chung thấp hơn hay cao hơn so với nút cha của nó? Nói chung là thấp hơn/cao hơn, hay luôn luôn thấp hơn/cao hơn?
3. Nếu cây quyết định bị quá khớp với tập dữ liệu huấn luyện, liệu có nên thử giảm kích thước của nó không? độ sâu tối đa?
4. Nếu cây quyết định không phù hợp với tập dữ liệu huấn luyện, liệu có nên thử điều chỉnh tỷ lệ (scaling) cây quyết định không? Các tính năng đầu vào?
5. Nếu mất một giờ để huấn luyện một cây quyết định trên tập dữ liệu huấn luyện chứa 1 triệu trường hợp, thì sẽ mất khoảng bao nhiêu thời gian để huấn luyện một cây quyết định khác trên tập dữ liệu huấn luyện chứa 10 triệu trường hợp? Gợi ý: hãy xem xét độ phức tạp tính toán của thuật toán CART.
6. Nếu mất một giờ để huấn luyện một cây quyết định trên một tập dữ liệu huấn luyện nhất định, thì sẽ mất khoảng bao nhiêu thời gian nếu bạn tăng gấp đôi số lượng đặc trưng?
7. Huấn luyện và tinh chỉnh Cây quyết định cho tập dữ liệu về các mặt trăng bằng cách làm theo các bước sau:
 - a. Sử dụng `make_moons(n_samples=10000, noise=0.4)` để tạo ra Bộ dữ liệu mặt trăng.
 - b. Sử dụng hàm `train_test_split()` để chia tập dữ liệu thành tập huấn luyện và tập kiểm tra.
 - c. Sử dụng tìm kiếm lưới với kiểm định chéo (với sự hỗ trợ của `GridSearchCV`) (lớp) để tìm các giá trị siêu tham số phù hợp cho `DecisionTreeClassifier`. Gợi ý: hãy thử các giá trị khác nhau cho `max_leaf_nodes`.
 - d. Huấn luyện mô hình trên toàn bộ tập dữ liệu huấn luyện bằng cách sử dụng các siêu tham số này, và đo lường hiệu suất của mô hình trên tập dữ liệu kiểm tra. Bạn sẽ đạt được độ chính xác khoảng 85% đến 87%.
8. Hãy trồng một khu rừng bằng cách làm theo các bước sau:
 - a. Tiếp tục bài tập trước, hãy tạo 1.000 tập con từ tập dữ liệu huấn luyện, mỗi tập chứa 100 trường hợp được chọn ngẫu nhiên. Gợi ý: bạn có thể sử dụng lớp `ShuffleSplit` của `Scikit-Learn` cho việc này.

b. Huấn luyện một cây quyết định trên mỗi tập con, sử dụng các giá trị siêu tham số tối ưu tìm được trong bài tập trước. Đánh giá 1.000 cây quyết định này trên tập dữ liệu thử nghiệm. Vì chúng được huấn luyện trên các tập dữ liệu nhỏ hơn, nên các cây quyết định này có thể sẽ hoạt động kém hơn cây quyết định đầu tiên, chỉ đạt độ chính xác khoảng 80%.

c. Giờ đến phần thú vị nhất. Với mỗi trường hợp trong tập dữ liệu kiểm thử, hãy tạo ra các dự đoán về... Tạo 1.000 cây quyết định và chỉ giữ lại dự đoán xuất hiện thường xuyên nhất (bạn có thể sử dụng hàm `mode()` của SciPy cho việc này). Cách tiếp cận này cung cấp cho bạn các dự đoán dựa trên đa số phiếu bầu trên tập dữ liệu thử nghiệm.

d. Đánh giá các dự đoán này trên tập dữ liệu kiểm thử: bạn sẽ nhận được kết quả cao hơn một chút. Độ chính xác cao hơn mô hình đầu tiên của bạn (cao hơn khoảng 0,5 đến 1,5%). Chúc mừng, bạn đã huấn luyện thành công bộ phân loại Rừng Ngẫu nhiên!

Đáp án cho các bài tập này có sẵn ở cuối tập tài liệu của chương này, tại địa chỉ <https://hom1.info/colab3>.

1. P là tập hợp các bài toán có thể giải được trong thời gian đa thức (tức là đa thức của kích thước tập dữ liệu). NP là tập hợp các bài toán mà lời giải của chúng có thể được kiểm chứng trong thời gian đa thức. Một bài toán NP-Hard là một bài toán có thể được quy về một bài toán NP-Hard đã biết trong thời gian đa thức. Một bài toán NP-Complete vừa là NP vừa là NP-Hard. Một câu hỏi toán học quan trọng chưa được giải đáp là liệu P có bằng NP hay không. Nếu $P = NP$ (điều này có vẻ hợp lý), thì sẽ không bao giờ tìm thấy thuật toán đa thức nào cho bất kỳ bài toán NP-Complete nào (ngoại trừ có lẽ một ngày nào đó trên máy tính lượng tử).

2. Xem bài phân tích thú vị của Sebastian Raschka [để biết thêm chi tiết](#).

[OceanofPDF.com](https://oceanofpdf.com)

Chương 7. Học tập kết hợp và Rừng ngẫu nhiên

Với sách điện tử phát hành sớm, bạn sẽ nhận được sách ở dạng sơ khai nhất-nội dung thô và chưa chỉnh sửa của tác giả khi họ viết-nhờ đó bạn có thể tận dụng những công nghệ này rất lâu trước khi các tựa sách được phát hành chính thức.

Đây sẽ là chương thứ 7 của cuốn sách cuối cùng. Kho lưu trữ GitHub là <https://github.com/ageron/handson-ml3>.

Nếu bạn có ý kiến đóng góp về cách chúng tôi có thể cải thiện nội dung và/hoặc ví dụ trong cuốn sách này, hoặc nếu bạn nhận thấy thiếu sót nào trong chương này, vui lòng liên hệ với biên tập viên theo địa chỉ mcronin@oreilly.com.

Giả sử bạn đặt một câu hỏi phức tạp cho hàng nghìn người ngẫu nhiên, sau đó tổng hợp câu trả lời của họ. Trong nhiều trường hợp, bạn sẽ thấy rằng câu trả lời tổng hợp này tốt hơn câu trả lời của chuyên gia. Điều này được gọi là trí tuệ đám đông. Tương tự, nếu bạn tổng hợp các dự đoán của một nhóm các công cụ dự đoán (chẳng hạn như bộ phân loại hoặc bộ hồi quy), bạn thường sẽ nhận được các dự đoán tốt hơn so với dự đoán của riêng công cụ dự đoán tốt nhất. Một nhóm các công cụ dự đoán được gọi là một tập hợp; do đó, kỹ thuật này được gọi là Học tập Tập hợp (Ensemble Learning), và thuật toán Học tập Tập hợp được gọi là phương pháp Tập hợp (Ensemble method).

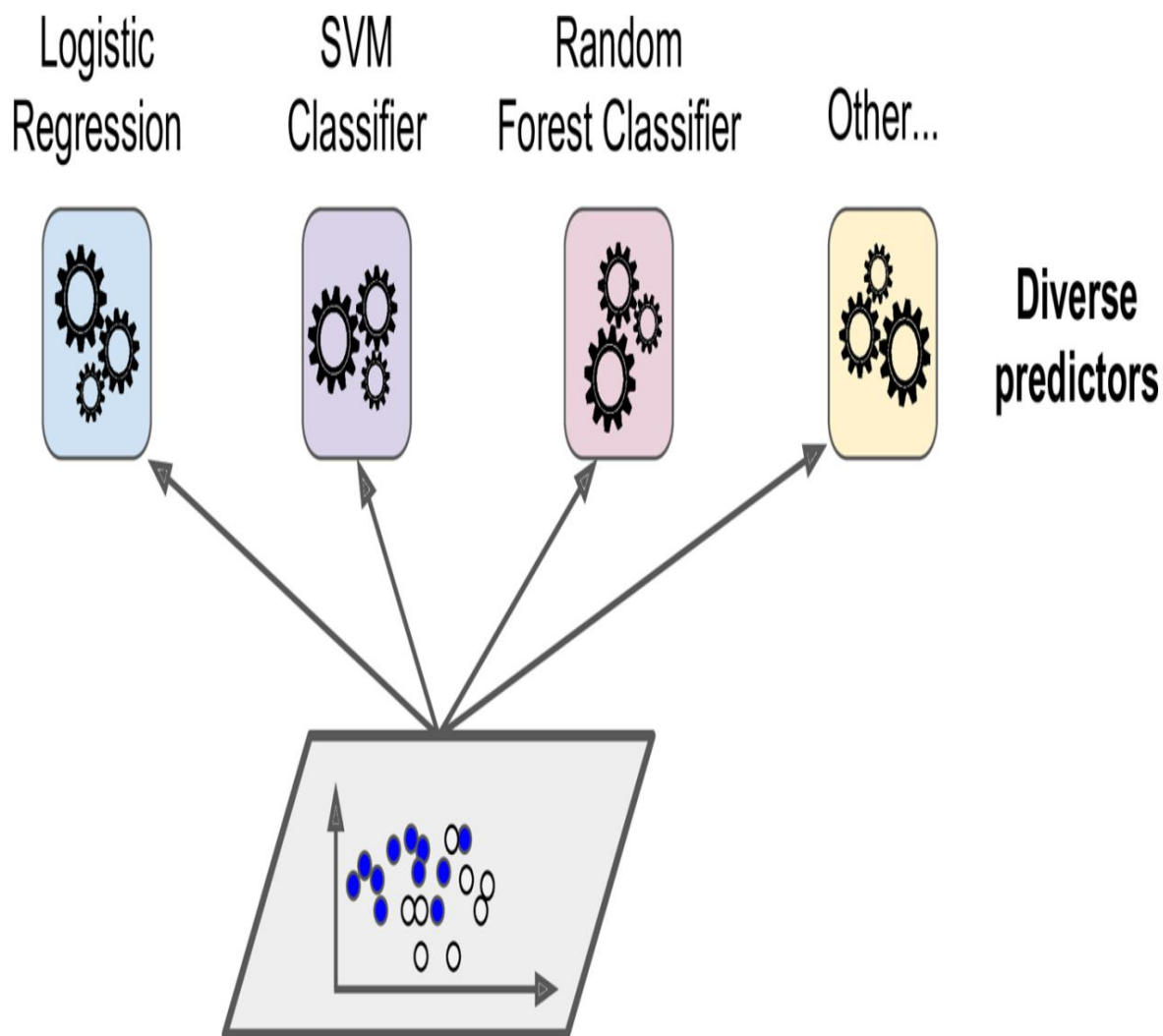
Ví dụ về phương pháp kết hợp (Ensemble), bạn có thể huấn luyện một nhóm các cây quyết định (Decision Tree), mỗi cây trên một tập con ngẫu nhiên khác nhau của tập dữ liệu huấn luyện. Sau đó, bạn có thể thu được dự đoán của tất cả các cây riêng lẻ, và lớp nhận được nhiều phiếu bầu nhất sẽ là dự đoán của toàn bộ nhóm (xem bài tập cuối cùng trong [Chương 6](#)). Một nhóm các cây quyết định như vậy được gọi là Rừng ngẫu nhiên (Random Forest), và mặc dù đơn giản, đây là một trong những thuật toán Học máy mạnh mẽ nhất hiện nay.

Như đã thảo luận trong [Chương 2](#), bạn sẽ thường sử dụng các phương pháp kết hợp (Ensemble methods) gần cuối dự án, sau khi đã xây dựng được một vài mô hình dự đoán tốt, để kết hợp chúng thành một mô hình dự đoán thậm chí còn tốt hơn. Trên thực tế, các giải pháp chiến thắng trong các cuộc thi Học máy thường liên quan đến một số phương pháp kết hợp – nổi tiếng nhất là trong [cuộc thi Netflix Prize](#).

Trong chương này, chúng ta sẽ thảo luận về các phương pháp Ensemble phổ biến nhất, bao gồm các thuật toán phân loại dựa trên bỏ phiếu, bagging, pasteing, Random Forests, boosting và stacking.

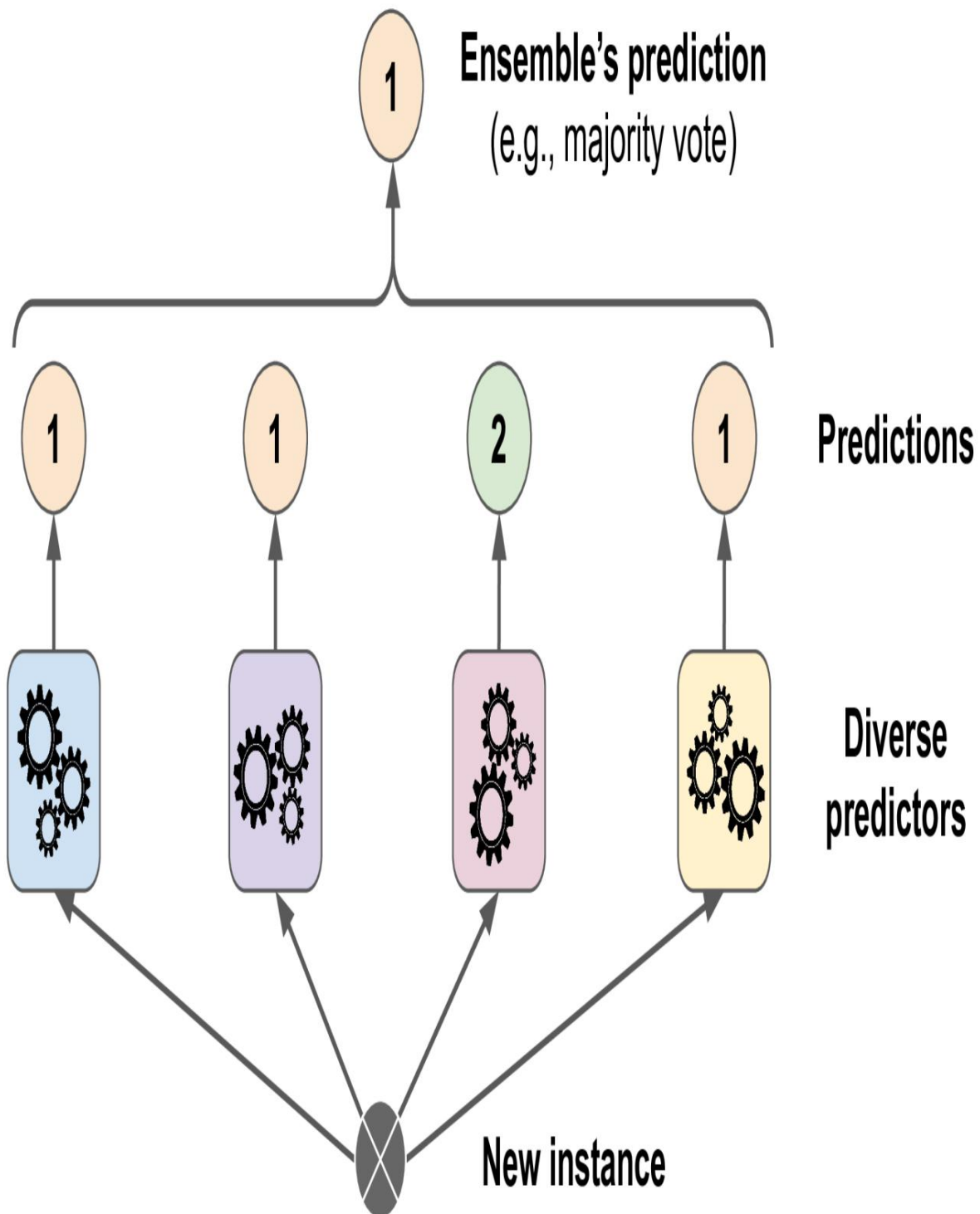
Bộ phân loại phiếu bầu

Giả sử bạn đã huấn luyện một vài bộ phân loại, mỗi bộ đạt độ chính xác khoảng 80%. Bạn có thể có một bộ phân loại Hồi quy Logistic, một bộ phân loại SVM, một bộ phân loại Rừng ngẫu nhiên, một bộ phân loại K-Hàng xóm gần nhất, và có thể một vài bộ khác nữa (xem [Hình 7-1](#)).



Hình 7-1. Huấn luyện các bộ phân loại đa dạng

Một cách rất đơn giản để tạo ra một bộ phân loại tốt hơn nữa là tổng hợp các dự đoán của từng bộ phân loại: lớp nhận được nhiều phiếu bầu nhất là dự đoán của toàn bộ hệ thống. Bộ phân loại bỏ phiếu theo đa số này được gọi là bộ phân loại bỏ phiếu cứng (xem [Hình 7-2](#)).



Hình 7-2. Dự đoán của bộ phân loại bỏ phiếu cứng

Điều khá bất ngờ là, bộ phân loại dựa trên bỏ phiếu này thường đạt độ chính xác cao hơn cả bộ phân loại tốt nhất trong toàn bộ hệ thống. Trên thực tế, ngay cả khi mỗi bộ phân loại là một bộ học yếu (nghĩa là nó chỉ hoạt động tốt hơn một chút so với dự đoán ngẫu nhiên), hệ thống vẫn có thể là một bộ học mạnh.

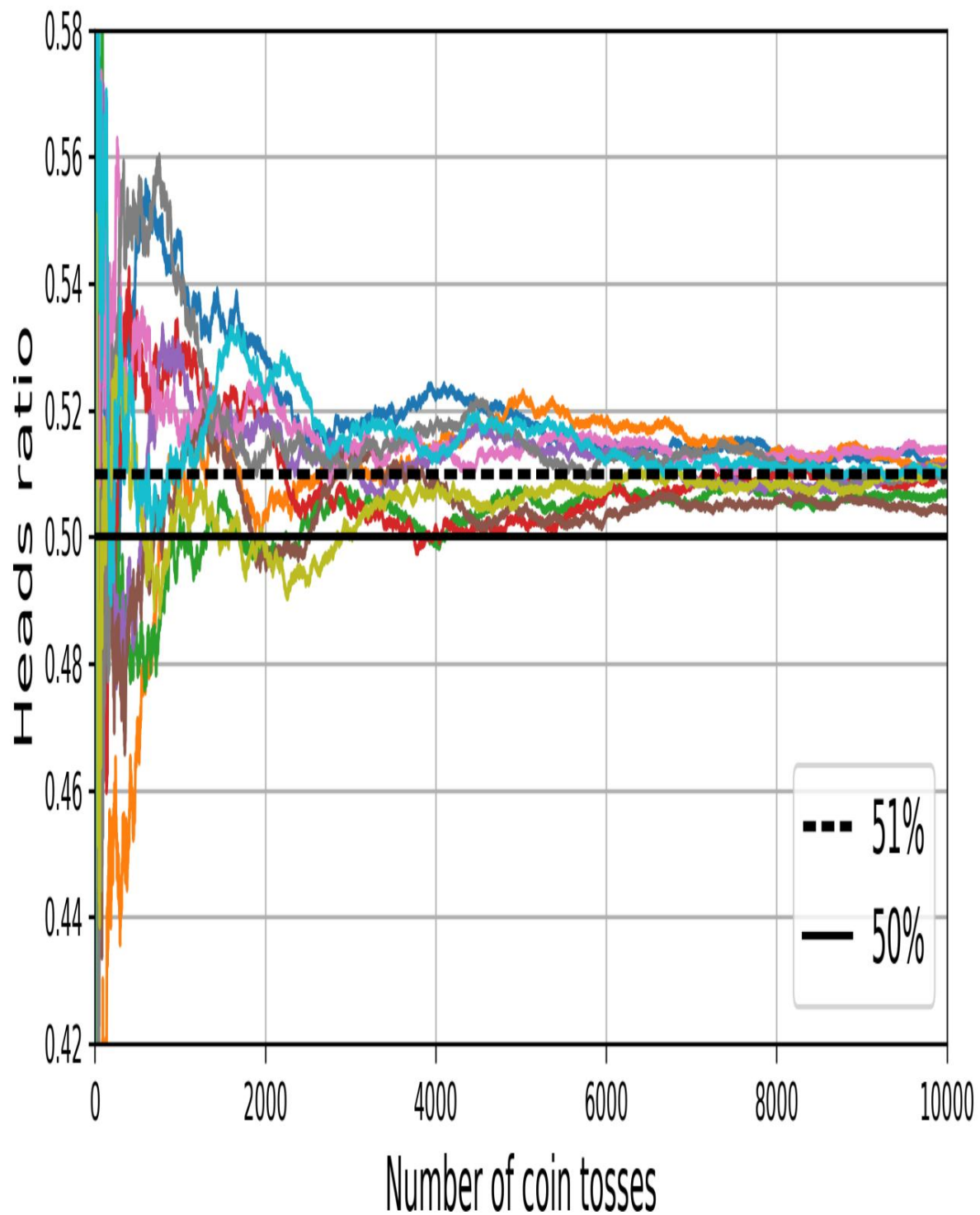
(đạt độ chính xác cao), với điều kiện là có đủ số lượng các mô hình học yếu trong tập hợp và chúng đủ đa dạng.

Làm sao điều này có thể xảy ra? Ví dụ sau đây có thể giúp làm sáng tỏ bí ẩn này.

Giả sử bạn có một đồng xu hơi không cân bằng, có 51% khả năng ra mặt ngửa và 49% khả năng ra mặt sấp. Nếu bạn tung đồng xu 1.000 lần, bạn thường sẽ nhận được khoảng 510 mặt ngửa và 490 mặt sấp, và do đó phần lớn sẽ là mặt ngửa. Nếu bạn tính toán, bạn sẽ thấy rằng xác suất nhận được phần lớn là mặt ngửa sau 1.000 lần tung là khoảng 75%.

Càng tung đồng xu nhiều lần, xác suất càng cao (ví dụ, với 10.000 lần tung, xác suất tăng lên trên 97%). Điều này là do định luật số lớn: khi bạn tiếp tục tung đồng xu, tỷ lệ xuất hiện mặt ngửa càng ngày càng tiến gần đến xác suất xuất hiện mặt ngửa (51%).

Hình 7-3 cho thấy 10 loạt tung đồng xu không cân bằng. Bạn có thể thấy rằng khi số lần tung tăng lên, tỷ lệ mặt ngửa tiến gần đến 51%. Cuối cùng, cả 10 loạt đều có tỷ lệ rất gần 51%, luôn cao hơn 50%.



Hình 7-3. Định luật số lớn

Tương tự, giả sử bạn xây dựng một tập hợp gồm 1.000 bộ phân loại, mỗi bộ chỉ chính xác 51% (gần như không hơn gì dự đoán ngẫu nhiên). Nếu bạn dự đoán lớp được đa số người dùng bình chọn, bạn có thể hy vọng đạt độ chính xác lên đến 75%! Tuy nhiên, điều này chỉ đúng nếu tất cả các bộ phân loại hoàn toàn độc lập, tạo ra các lỗi không tương quan, điều này rõ ràng là không thể xảy ra.

Nguyên nhân là do chúng được huấn luyện trên cùng một tập dữ liệu. Chúng có khả năng mắc cùng loại lỗi, do đó sẽ có nhiều phiếu bầu đa số cho lớp sai, làm giảm độ chính xác của mô hình kết hợp.

MẸO

Các phương pháp kết hợp hoạt động tốt nhất khi các mô hình dự đoán càng độc lập với nhau càng tốt. Một cách để có được các mô hình phân loại đa dạng là huấn luyện chúng bằng các thuật toán rất khác nhau. Điều này làm tăng khả năng chúng sẽ mắc các loại lỗi rất khác nhau, cải thiện độ chính xác của mô hình kết hợp.

Scikit-Learn cung cấp lớp `VotingClassifier` khá dễ sử dụng: chỉ cần cung cấp cho nó một danh sách các cặp tên/biến dự đoán, và sử dụng nó như một bộ phân loại thông thường, vậy là xong! Hãy thử nghiệm trên tập dữ liệu moons (được giới thiệu trong [Chương 6](#)): chúng ta sẽ tải và chia tập dữ liệu moons thành tập huấn luyện và tập kiểm tra, sau đó chúng ta sẽ tạo và huấn luyện một bộ phân loại bỏ phiếu bao gồm ba bộ phân loại khác nhau:

```
from sklearn.datasets import make_moons from
sklearn.ensemble import RandomForestClassifier, VotingClassifier from sklearn.linear_model
import LogisticRegression from sklearn.model_selection import
train_test_split from sklearn.svm import SVC

X, y = make_moons(n_samples=500, noise=0.30, random_state=42)
X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=42)

voting_clf = VotingClassifier( estimators=[

    ('lr', LogisticRegression(random_state=42)), ('rf',
    RandomForestClassifier(random_state=42)), ('svc',
    SVC(random_state=42))

])

voting_clf.fit(X_train, y_train)
```

Khi bạn huấn luyện một `VotingClassifier`, nó sẽ sao chép mọi bộ ước lượng và huấn luyện các bản sao đó. Các bộ ước lượng gốc có sẵn thông qua thuộc tính `estimators`, trong khi các bản sao đã được huấn luyện có sẵn thông qua thuộc tính `estimators_`. Nếu bạn muốn sử dụng từ điển thay vì danh sách, bạn có thể sử dụng `named_estimators` hoặc `named_estimators_` thay thế. Ví dụ, hãy xem độ chính xác của mỗi bộ phân loại đã được huấn luyện trên tập dữ liệu thử nghiệm:

```
>>> for name, clf in voting_clf.named_estimators_.items(): print(name, "=",
...     clf.score(X_test, y_test))
...
lr = 0,864
rf = 0,896
svc = 0,896
```

Khi bạn gọi phương thức `predict()` của bộ phân loại bỏ phiếu, nó sẽ thực hiện bỏ phiếu cứng. Ví dụ, bộ phân loại bỏ phiếu dự đoán lớp 1 cho trường hợp đầu tiên của tập dữ liệu thử nghiệm, bởi vì 2 trong số 3 bộ phân loại dự đoán lớp đó:

```
>>> voting_clf.predict(X_test[:1])
array([1])
>>> [clf.predict(X_test[:1]) for clf in voting_clf.estimators_]
[array([1]), array([1]), array([0])]
```

Bây giờ chúng ta hãy xem xét hiệu suất của bộ phân loại bỏ phiếu trên tập dữ liệu thử nghiệm:

```
>>> voting_clf.score(X_test, y_test) 0.912
```

Vậy đây! Thuật toán phân loại dựa trên bỏ phiếu vượt trội hơn tất cả các thuật toán phân loại riêng lẻ.

Nếu tất cả các bộ phân loại đều có khả năng ước tính xác suất lớp (tức là tất cả đều có phương thức `predict_proba()`), thì bạn có thể yêu cầu Scikit-Learn dự đoán lớp có xác suất lớp cao nhất, được tính trung bình trên tất cả các bộ phân loại riêng lẻ. Điều này được gọi là bỏ phiếu mềm. Nó thường đạt được hiệu suất cao hơn so với bỏ phiếu cứng vì nó coi trọng hơn các phiếu bầu có độ tin cậy cao. Tất cả những gì bạn cần làm là đặt siêu tham số bỏ phiếu của bộ phân loại thành "soft" và đảm bảo rằng tất cả các bộ phân loại đều có thể ước tính xác suất lớp.

Theo mặc định, lớp SVC không hoạt động như vậy, vì vậy bạn cần đặt siêu tham số xác suất của nó thành True (điều này sẽ khiến lớp SVC sử dụng phương pháp kiểm định chéo để ước tính xác suất của lớp, làm chậm quá trình huấn luyện, và nó sẽ thêm phương thức `predict_proba()`). Hãy thử xem:

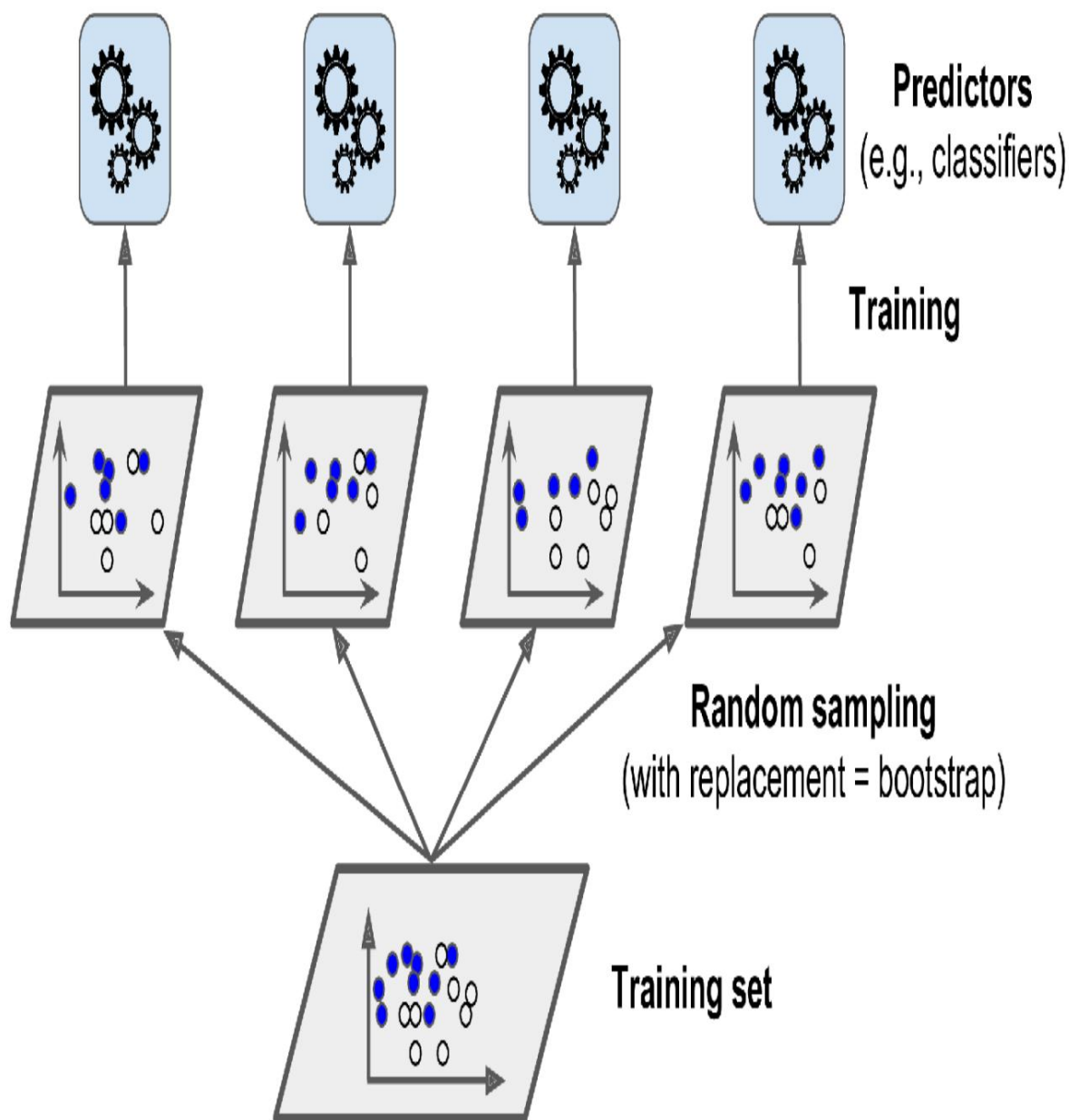
```
>>> voting_clf.voting = "soft" >>>
voting_clf.named_estimators["svc"].probability = True >>> voting_clf.fit(X_train,
y_train) >>> voting_clf.score(X_test, y_test) 0.92
```

Chúng ta đạt được độ chính xác 92% chỉ bằng cách sử dụng phương pháp bỏ phiếu mềm, không tồi chút nào!

Đóng gói và dán

Một cách để có được một tập hợp các bộ phân loại đa dạng là sử dụng các thuật toán huấn luyện rất khác nhau, như đã thảo luận ở trên. Một cách tiếp cận khác là sử dụng cùng một thuật toán huấn luyện cho mọi bộ dự đoán nhưng huấn luyện chúng trên các tập con ngẫu nhiên khác nhau của tập dữ liệu huấn luyện. Khi lấy mẫu được thực hiện có hoàn trả, phương pháp này được gọi là **bagging**. (viết tắt của bootstrap aggregating - tổng hợp bootstrap).³ Khi việc lấy mẫu được thực hiện mà không thay thế, nó được gọi là **lấy mẫu dán**.⁴

Nói cách khác, cả hai phương pháp bagging và pasteing đều cho phép lấy mẫu các trường hợp huấn luyện nhiều lần trên nhiều biến dự đoán khác nhau, nhưng chỉ bagging mới cho phép lấy mẫu các trường hợp huấn luyện nhiều lần cho cùng một biến dự đoán. Quá trình lấy mẫu và huấn luyện này được thể hiện trong **Hình 7-4**.



Hình 7-4. Phương pháp bagging và pasteing bao gồm việc huấn luyện một số mô hình dự đoán trên các mẫu ngẫu nhiên khác nhau của tập dữ liệu huấn luyện.

Sau khi tất cả các mô hình dự đoán được huấn luyện, hệ thống có thể đưa ra dự đoán cho một trường hợp mới bằng cách đơn giản là tổng hợp các dự đoán của tất cả các mô hình dự đoán. Hàm tổng hợp thường là giá trị phổ biến nhất trong phân loại (tức là dự đoán xuất hiện thường xuyên nhất, giống như bộ phân loại bỏ phiếu cứng), hoặc giá trị trung bình trong hồi quy. Mỗi mô hình dự đoán riêng lẻ có độ lệch cao hơn so với khi được huấn luyện trên tập dữ liệu huấn luyện ban đầu, nhưng việc tổng hợp làm giảm cả độ lệch và độ chính xác.

⁵ sự biến thiên. Nhìn chung, kết quả cuối cùng là tập hợp các mô hình có độ lệch tương tự nhưng sự biến thiên thấp hơn so với một mô hình dự đoán đơn lẻ được huấn luyện trên tập dữ liệu huấn luyện ban đầu.

Như bạn có thể thấy trong **Hình 7-4**, tất cả các mô hình dự đoán đều có thể được huấn luyện song song, thông qua các lõi CPU khác nhau hoặc thậm chí các máy chủ khác nhau. Tương tự, các dự đoán cũng có thể được thực hiện song song. Đây là một trong những lý do tại sao bagging và pasteing lại phổ biến đến vậy: chúng có khả năng mở rộng rất tốt.

Đóng gói và dán trong Scikit-Learn

Scikit-Learn cung cấp một API đơn giản cho cả phương pháp bagging và pasteing với lớp

`BaggingClassifier` (hoặc `BaggingRegressor` cho hồi quy). Đoạn mã sau huấn luyện một tập hợp gồm 500 bộ phân loại Cây quyết định: mỗi bộ được huấn luyện trên 100 mẫu huấn luyện được lấy ngẫu nhiên từ tập huấn luyện có hoàn trả (⁶ đây là một ví dụ về bagging, nhưng nếu bạn muốn sử dụng pasteing, chỉ cần đặt `bootstrap=False`). Tham số `n_jobs` cho Scikit-Learn biết số lõi CPU cần sử dụng cho huấn luyện và dự đoán, và `-1` cho Scikit-Learn biết sử dụng tất cả các lõi có sẵn.

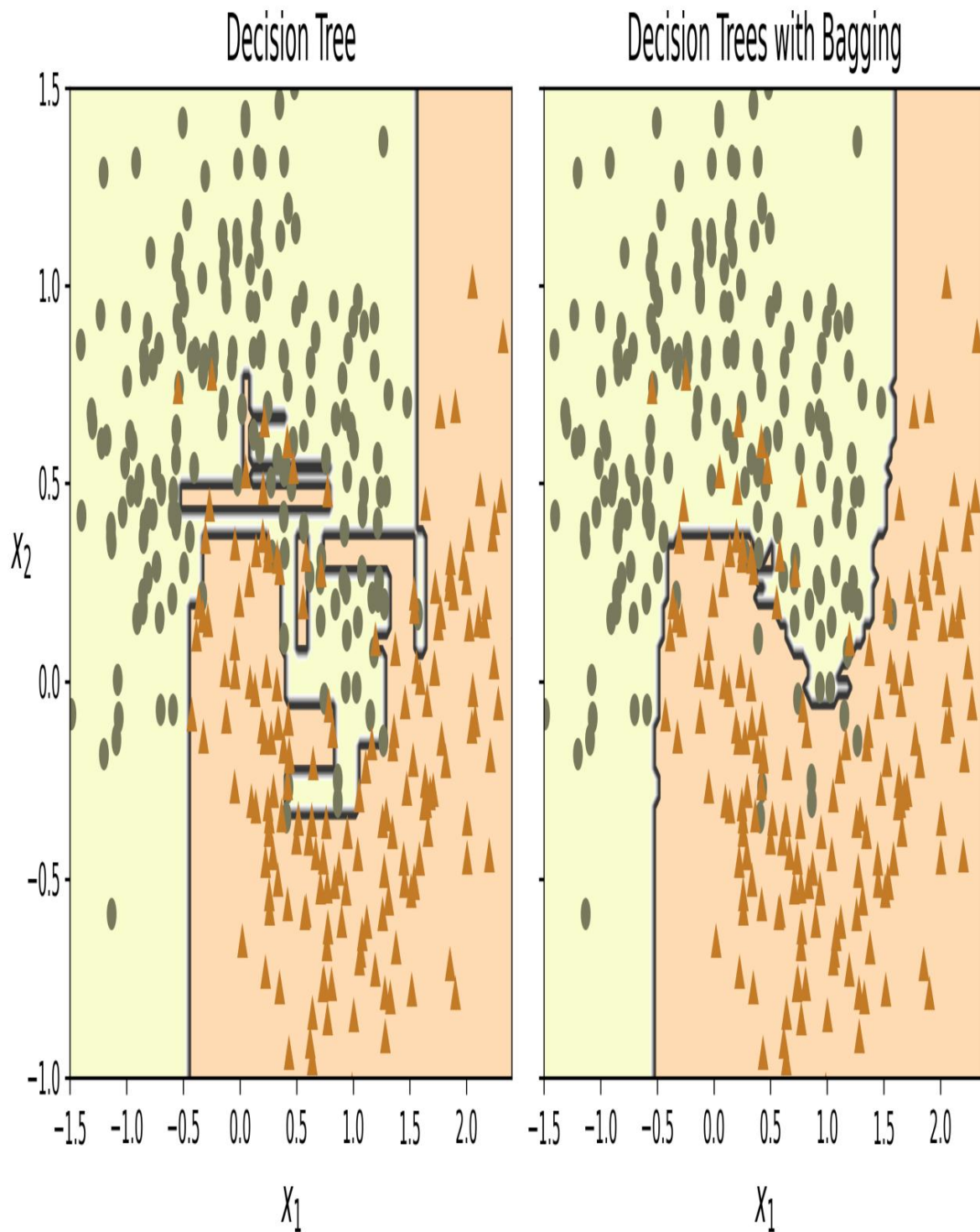
```
from sklearn.ensemble import BaggingClassifier from
sklearn.tree import DecisionTreeClassifier

bag_clf = BaggingClassifier(DecisionTreeClassifier(), n_estimators=500,
                           max_samples=100, n_jobs=-1, random_state=42)
bag_clf.fit(X_train, y_train)
```

GHI CHÚ

`BaggingClassifier` tự động thực hiện bỏ phiếu mềm thay vì bỏ phiếu cứng nếu bộ phân loại cơ sở có thể ước tính xác suất lớp (tức là nếu nó có phương thức `predict_proba()`), điều này đúng với các bộ phân loại Cây quyết định.

Hình 7-5 so sánh ranh giới quyết định của một Cây Quyết định đơn lẻ với ranh giới quyết định của một tập hợp bagging gồm 500 cây (từ đoạn mã trước đó), cả hai đều được huấn luyện trên tập dữ liệu moons. Như bạn có thể thấy, dự đoán của tập hợp có khả năng khái quát hóa tốt hơn nhiều so với dự đoán của một Cây Quyết định đơn lẻ: tập hợp có độ lệch tương đương nhưng phương sai nhỏ hơn (nó mắc số lỗi gần như nhau trên tập huấn luyện, nhưng ranh giới quyết định ít bất thường hơn).



Hình 7-5. Một cây quyết định đơn lẻ (bên trái) so với một tập hợp gồm 500 cây (bên phải).

Phương pháp bagging mang lại sự đa dạng hơn một chút trong các tập con mà mỗi mô hình dự đoán được huấn luyện, do đó bagging có độ lệch cao hơn một chút so với phương pháp paste; nhưng sự đa dạng bổ sung này cũng có nghĩa là các mô hình dự đoán ít tương quan hơn, do đó phương sai của mô hình tổng hợp được giảm xuống.

Nhìn chung, việc sử dụng kỹ thuật bagging thường tạo ra những mô hình tốt hơn, điều này giải thích tại sao nó thường được sử dụng rộng rãi.

Phương pháp này được ưa chuộng hơn. Tuy nhiên, nếu bạn có thời gian rảnh và đủ sức mạnh xử lý CPU, bạn có thể sử dụng phương pháp kiểm định chéo để đánh giá cả hai phương pháp bagging và pasteing, rồi chọn phương pháp nào hoạt động tốt nhất.

Đánh giá ngoài túi

Với phương pháp bagging, một số mẫu dữ liệu huấn luyện có thể được lấy mẫu nhiều lần cho bất kỳ biến dự đoán nào, trong khi những mẫu khác có thể không được lấy mẫu. Theo mặc định, BaggingClassifier lấy mẫu m mẫu dữ liệu huấn luyện có hoàn trả (bootstrap=True), trong đó m là kích thước của tập dữ liệu huấn luyện. Với quy trình này, có thể chứng minh bằng toán học rằng trung bình chỉ khoảng 63% mẫu dữ liệu huấn luyện được lấy mẫu cho mỗi biến dự đoán. 37% mẫu dữ liệu huấn luyện còn lại không được lấy mẫu được gọi là các mẫu dữ liệu ngoài bag (out-of-bag - oob). Lưu ý rằng 37% này không giống nhau đối với tất cả các biến dự đoán.

Một mô hình kết hợp bagging có thể được đánh giá bằng cách sử dụng các trường hợp oob (ngoài phạm vi) mà không cần đến một tập dữ liệu xác thực riêng biệt: thực tế, nếu có đủ các mô hình ước lượng, thì mỗi trường hợp trong tập huấn luyện có khả năng là một trường hợp oob của một số mô hình ước lượng, vì vậy các mô hình ước lượng này có thể được sử dụng để đưa ra dự đoán kết hợp công bằng cho trường hợp đó. Sau khi có dự đoán cho mỗi trường hợp, bạn có thể tính toán độ chính xác dự đoán của mô hình kết hợp (hoặc bất kỳ chỉ số nào khác).

Trong Scikit-Learn, bạn có thể đặt `oob\_score=True` khi tạo `BaggingClassifier`

để yêu cầu đánh giá OOB tự động sau khi huấn luyện. Đoạn mã sau đây minh họa điều này. Điểm đánh giá thu được sẽ hiển thị trong thuộc tính `oob\_score\_`:

```
>>> bag_clf = BaggingClassifier(DecisionTreeClassifier(), n_estimators=500,
...                             oob_score=True, n_jobs=-1, random_state=42)
...
>>> bag_clf.fit(X_train, y_train) >>>
bag_clf.oob_score_ 0.896
```

Theo đánh giá oob này, BaggingClassifier có khả năng đạt độ chính xác khoảng 89,6% trên tập dữ liệu thử nghiệm.

Hãy cùng kiểm chứng điều này:

```
>>> from sklearn.metrics import accuracy_score >>> y_pred =
bag_clf.predict(X_test) >>> accuracy_score(y_test,
y_pred) 0.92
```

Chúng tôi đạt độ chính xác 92% trong bài kiểm tra. Đánh giá ban đầu hơi bi quan, thấp hơn khoảng 2%.

Hàm quyết định oob cho mỗi trường hợp huấn luyện cũng có sẵn thông qua thuộc tính

`oob_decision_function_`. Vì bộ ước lượng cơ bản có phương thức `predict_proba()`, nên hàm

quyết định trả về xác suất lớp cho

mỗi trường hợp huấn luyện. Ví dụ, đánh giá oob ước tính rằng trường hợp huấn luyện đầu tiên có 67,6% xác suất thuộc về lớp tích cực và 32,4% xác suất thuộc về lớp tiêu cực:

```
>>> bag_clf.oob_decision_function_[:3] # xác suất cho 3 trường hợp đầu tiên
array([[0.32352941,
        0.67647059], [0.3375 ], [1. ]])
      , 0.6625
      , 0.
```

Các mảng ngẫu nhiên và các không gian con ngẫu nhiên

Lớp BaggingClassifier cũng hỗ trợ lấy mẫu các đặc trưng. Việc lấy mẫu được điều khiển bởi hai siêu tham số: max_features và bootstrap_features.

Chúng hoạt động tương tự như max_samples và bootstrap, nhưng dùng để lấy mẫu đặc trưng thay vì lấy mẫu cá thể. Do đó, mỗi mô hình dự đoán sẽ được huấn luyện trên một tập con ngẫu nhiên của các đặc trưng đầu vào.

Kỹ thuật này đặc biệt hữu ích khi bạn xử lý các dữ liệu đầu vào có chiều cao (như hình ảnh), vì nó có thể tăng tốc đáng kể quá trình huấn luyện. Lấy mẫu cả quá trình huấn luyện

Các trường hợp và tính năng được gọi là phương pháp **Vá lỗi ngẫu nhiên**⁸. Giữ lại tất cả các trường hợp huấn luyện (bằng cách đặt bootstrap=False và max_samples=1.0) nhưng lấy mẫu các đặc trưng (bằng cách đặt bootstrap_features thành True và/hoặc max_features thành một giá trị nhỏ hơn 1.0) được gọi là phương pháp **Không gian con ngẫu nhiên**⁹.

Việc lấy mẫu các đặc điểm giúp tăng thêm tính đa dạng cho các biến dự đoán, đánh đổi một chút sai lệch để có được phương sai thấp hơn.

Rừng ngẫu nhiên

Như chúng ta đã thảo luận, đó là một **Rừng ngẫu nhiên**¹⁰. là một tập hợp các Cây Quyết định, thường được huấn luyện bằng phương pháp bagging (hoặc đôi khi là pasteing), thường với max_samples được đặt bằng kích thước của tập huấn luyện. Thay vì xây dựng một BaggingClassifier và truyền cho nó một DecisionTreeClassifier, bạn có thể sử dụng lớp RandomForestClassifier, thuận tiện hơn và được tối ưu hóa cho Cây Quyết định (tương tự, có một lớp RandomForestRegressor cho các tác vụ hồi quy). Đoạn mã sau huấn luyện một bộ phân loại Rừng Ngẫu nhiên với 500 cây, mỗi cây giới hạn tối đa 16 nút và sử dụng tất cả các lõi CPU có sẵn:

```
from sklearn.ensemble import RandomForestClassifier

rnd_clf = RandomForestClassifier(n_estimators=500, max_leaf_nodes=16,
                                n_jobs=-1, random_state=42)
rnd_clf.fit(X_train, y_train)
```

```
y_pred_rf = rnd_clf.predict(X_test)
```

Với một vài ngoại lệ, RandomForestClassifier có tất cả các siêu tham số của DecisionTreeClassifier (để kiểm soát cách cây được tạo ra), cộng thêm tất cả các siêu tham số của BaggingClassifier để kiểm soát chính tập hợp các mô hình.

Thuật toán Rừng Ngẫu nhiên (Random Forest) đưa thêm yếu tố ngẫu nhiên vào quá trình xây dựng cây; thay vì tìm kiếm đặc trưng tốt nhất khi phân tách một nút (xem [Chương 6](#)), nó tìm kiếm đặc trưng tốt nhất trong một tập con ngẫu nhiên các đặc trưng. Theo mặc định, nó lấy mẫu $\sqrt{n}$ đặc trưng (trong đó n là tổng số đặc trưng).

Thuật toán này tạo ra sự đa dạng cây lớn hơn, điều này (một lần nữa) đánh đổi độ lệch cao hơn để có phương sai thấp hơn, nói chung mang lại một mô hình tổng thể tốt hơn. Vì vậy, BaggingClassifier sau đây tương đương với RandomForestClassifier trước đó:

```
bag_clf = BaggingClassifier(
    DecisionTreeClassifier(max_features="sqrt", max_leaf_nodes=16),
    n_estimators=500, n_jobs=-1, random_state=42)
```

Extra-Trees

Khi bạn xây dựng một cây trong Rừng Ngẫu nhiên, tại mỗi nút chỉ một tập hợp con ngẫu nhiên các đặc trưng được xem xét để phân tách (như đã thảo luận trước đó). Có thể làm cho cây trở nên ngẫu nhiên hơn nữa bằng cách sử dụng các ngưỡng ngẫu nhiên cho mỗi đặc trưng thay vì tìm kiếm các ngưỡng tốt nhất có thể (như Cây Quyết định thông thường). Để làm điều này, chỉ cần đặt `splitter="random"` khi tạo một DecisionTreeClassifier.

Một khu rừng gồm những cây có cấu trúc phân bố cực kỳ ngẫu nhiên được gọi là Rừng [Cây Phân Bố Cực Kỳ Ngẫu Nhiên](#) ¹² ([Extremely Randomized Trees](#)). Kết hợp (hay gọi tắt là Extra-Trees). Một lần nữa, kỹ thuật này đánh đổi độ lệch lớn hơn để có phương sai thấp hơn. Nó cũng giúp huấn luyện Extra-Trees nhanh hơn nhiều so với Rừng ngẫu nhiên thông thường, bởi vì việc tìm ra ngưỡng tốt nhất có thể cho mỗi đặc trưng tại mỗi nút là một trong những nhiệm vụ tốn nhiều thời gian nhất khi xây dựng cây.

Bạn có thể tạo bộ phân loại Extra-Trees bằng cách sử dụng lớp ExtraTreesClassifier của Scikit-Learn. API của nó giống hệt với lớp

RandomForestClassifier, ngoại trừ tham số bootstrap mặc định là False. Tương tự, lớp ExtraTreesRegressor có cùng API với lớp RandomForestRegressor, ngoại trừ tham số bootstrap mặc định là False.

MẸO

Rất khó để biết trước liệu RandomForestClassifier sẽ hoạt động tốt hơn hay kém hơn ExtraTreesClassifier. Thông thường, cách duy nhất để biết là thử cả hai và so sánh chúng. Kiểm định chéo.

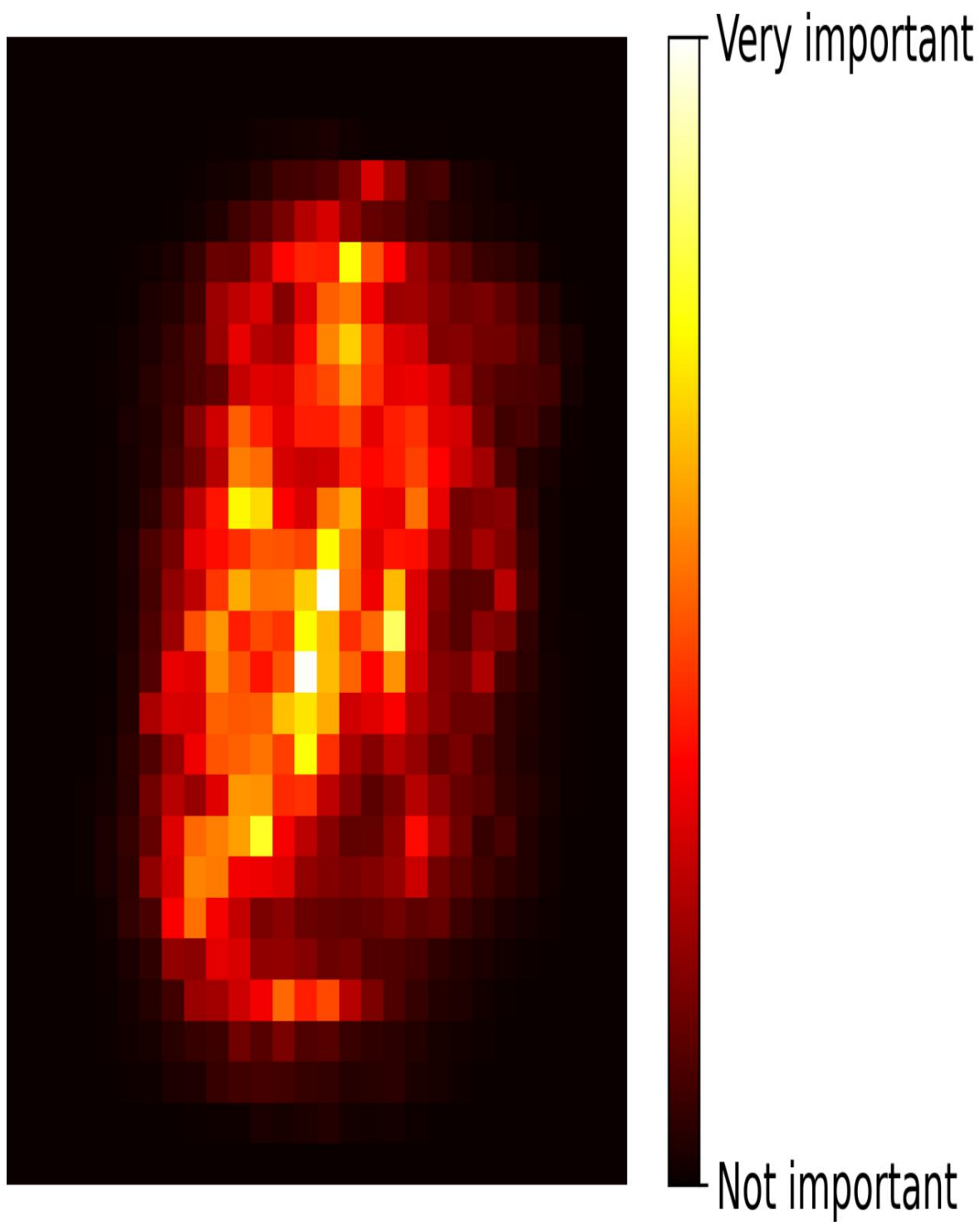
Tầm quan trọng của tính năng

Một ưu điểm tuyệt vời khác của Rừng Ngẫu nhiên là chúng giúp dễ dàng đo lường tầm quan trọng tương đối của từng đặc trưng. Scikit-Learn đo lường tầm quan trọng của một đặc trưng bằng cách xem xét mức độ giảm độ không thuần khiết trung bình mà các nút cây sử dụng đặc trưng đó đạt được trên tất cả các cây trong rừng. Chính xác hơn, đó là một trung bình có trọng số, trong đó trọng số của mỗi nút bằng với số lượng mẫu huấn luyện liên kết với nó (xem [Chương 6](#)).

Scikit-Learn tự động tính toán điểm số này cho từng đặc trưng sau khi huấn luyện, sau đó nó chuẩn hóa kết quả sao cho tổng tầm quan trọng bằng 1. Bạn có thể truy cập kết quả bằng biến `feature_importances_`. Ví dụ, đoạn mã sau huấn luyện một RandomForestClassifier trên tập dữ liệu iris (được giới thiệu trong [Chương 4](#)) và xuất ra tầm quan trọng của từng đặc trưng. Có vẻ như các đặc trưng quan trọng nhất là chiều dài cánh hoa (44%) và chiều rộng (42%), trong khi chiều dài và chiều rộng đài hoa lại không quan trọng lắm (lần lượt là 11% và 2%):

```
>>> from sklearn.datasets import load_iris >>> iris =
load_iris(as_frame=True) >>> rnd_clf =
RandomForestClassifier(n_estimators=500, random_state=42) >>> rnd_clf.fit(iris.data, iris.target)
>>> for score, name in zip(rnd_clf.feature_importances_,
iris.data.columns):
...     print(round(score, 2), name)
...
0,11 chiều dài đài hoa (cm)
0,02 chiều rộng đài hoa (cm)
0,44 chiều dài cánh hoa (cm)
0,42 chiều rộng cánh hoa (cm)
```

Tương tự, nếu bạn huấn luyện một bộ phân loại Rừng ngẫu nhiên trên tập dữ liệu MNIST (được giới thiệu trong [Chương 3](#)) và vẽ biểu đồ tầm quan trọng của từng pixel, bạn sẽ nhận được hình ảnh được thể hiện trong [Hình 7-6](#).



Hình 7-6. Độ quan trọng của pixel MNIST (theo phân loại Rừng ngẫu nhiên)

Rừng ngẫu nhiên rất hữu ích để nhanh chóng hiểu được những đặc điểm nào thực sự quan trọng, đặc biệt nếu bạn cần thực hiện chọn lọc đặc điểm.

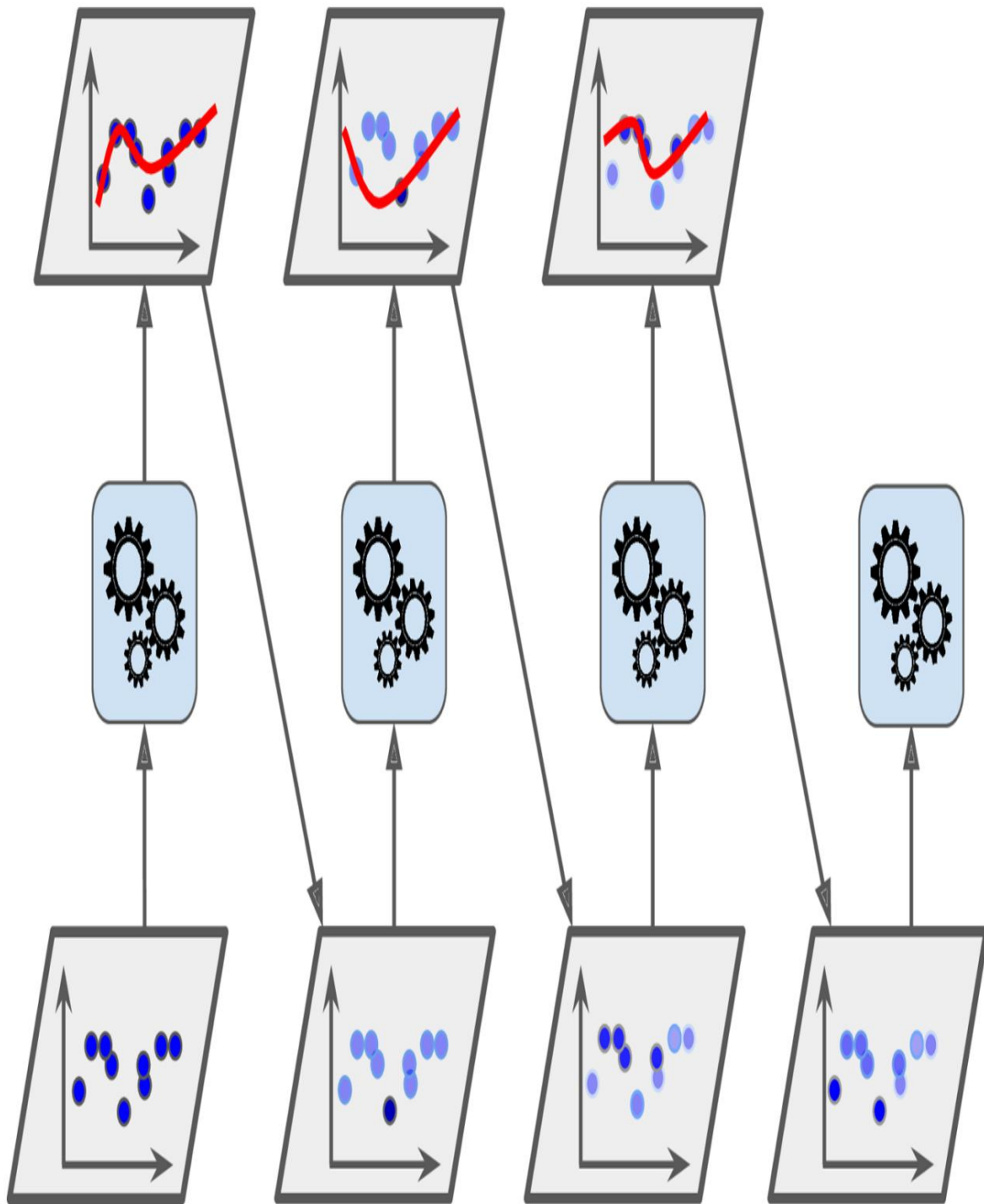
Tăng cường

Boosting (ban đầu được gọi là hypothesis boosting) đề cập đến bất kỳ phương pháp Ensemble nào có thể kết hợp nhiều mô hình học yếu thành một mô hình học mạnh. Ý tưởng chung của hầu hết các phương pháp boosting là huấn luyện các mô hình dự đoán theo trình tự, mỗi mô hình cố gắng sửa lỗi cho mô hình trước đó. Có rất nhiều phương pháp boosting khác nhau, nhưng phổ biến nhất cho đến nay là AdaBoost¹³. (viết tắt của Adaptive Boosting) và Gradient Boosting. Chúng ta hãy bắt đầu với AdaBoost.

AdaBoost

Một cách để mô hình dự đoán mới sửa lỗi cho mô hình tiền nhiệm là chú trọng hơn đến các trường hợp huấn luyện mà mô hình tiền nhiệm đã không khớp hoàn toàn. Điều này dẫn đến việc các mô hình dự đoán mới tập trung nhiều hơn vào các trường hợp khó. Đây là kỹ thuật được AdaBoost sử dụng.

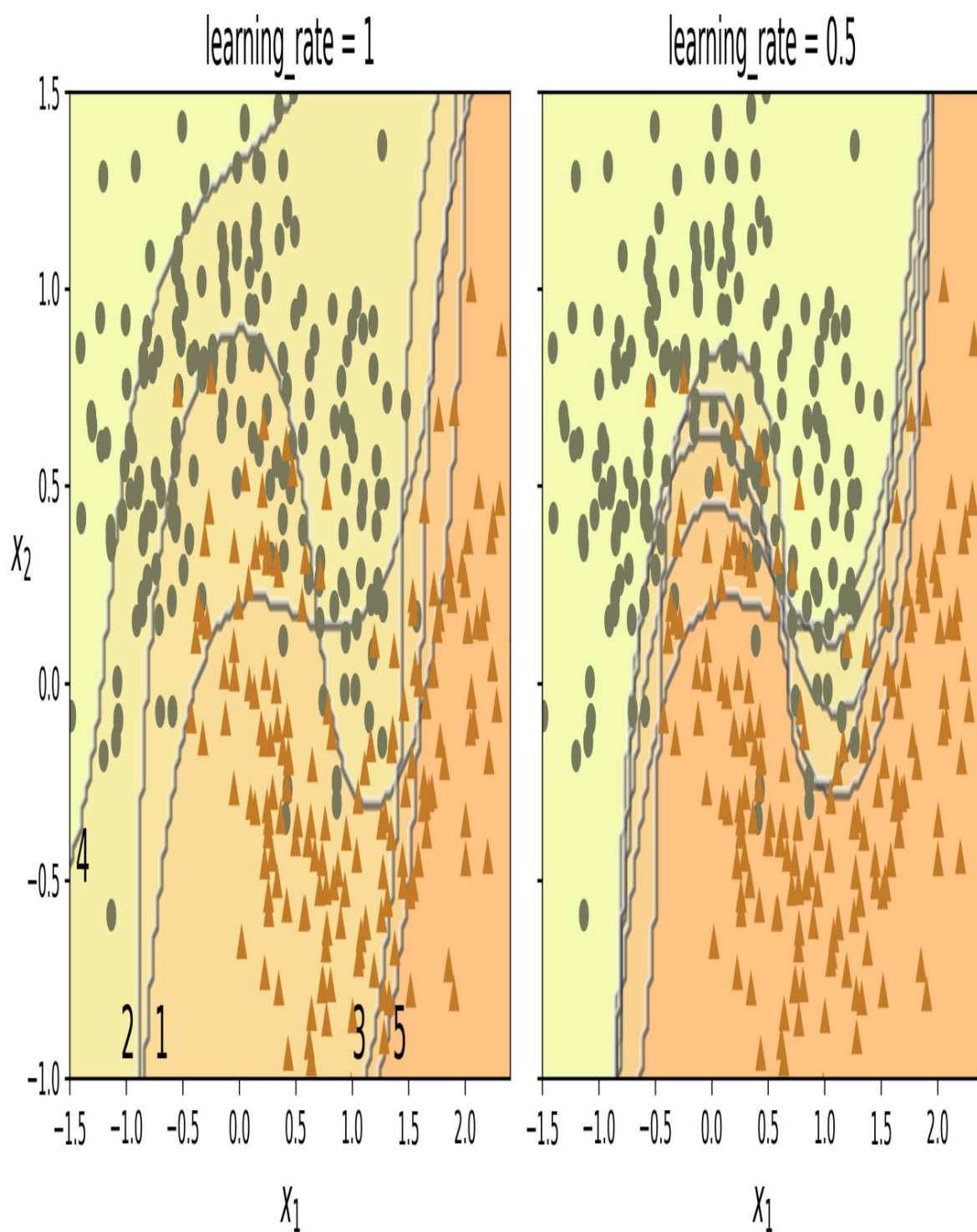
Ví dụ, khi huấn luyện bộ phân loại AdaBoost, thuật toán trước tiên huấn luyện một bộ phân loại cơ sở (chẳng hạn như Cây quyết định) và sử dụng nó để đưa ra dự đoán trên tập dữ liệu huấn luyện. Sau đó, thuật toán tăng trọng số tương đối của các trường hợp huấn luyện bị phân loại sai. Tiếp theo, nó huấn luyện một bộ phân loại thứ hai, sử dụng các trọng số đã cập nhật, và lại đưa ra dự đoán trên tập dữ liệu huấn luyện, cập nhật trọng số của các trường hợp, v.v. (xem Hình 7-7).



Hình 7-7. Huấn luyện tuần tự AdaBoost với cập nhật trọng số mẫu

Hình 7-8 thể hiện ranh giới quyết định của năm bộ dự đoán liên tiếp trên tập dữ liệu mặt trăng (trong ví dụ này, mỗi bộ dự đoán là một bộ phân loại SVM được điều chỉnh cao với hạt nhân RBF). Bộ phân loại đầu tiên dự đoán sai nhiều trường hợp, do đó trọng số của chúng được tăng lên. Vì vậy, bộ phân loại thứ hai sẽ xử lý tốt hơn các trường hợp này, và cứ thế tiếp tục. Đồ thị bên phải thể hiện cùng một chuỗi các biến dự đoán, ngoại trừ quá trình học tập.

Tỷ lệ lỗi giảm đi một nửa (tức là trọng số của các trường hợp bị phân loại sai được tăng lên ít hơn nhiều ở mỗi lần lặp). Như bạn thấy, kỹ thuật học tuần tự này có một số điểm tương đồng với Gradient Descent, ngoại trừ việc thay vì điều chỉnh các tham số của một bộ dự đoán duy nhất để giảm thiểu hàm chi phí, AdaBoost thêm các bộ dự đoán vào tập hợp, dần dần làm cho nó tốt hơn.



Hình 7-8. Ranh giới quyết định của các yếu tố dự đoán liên tiếp

Sau khi tất cả các mô hình dự đoán được huấn luyện, mô hình kết hợp sẽ đưa ra dự đoán tương tự như phương pháp bagging hoặc pasteing, ngoại trừ việc các mô hình dự đoán có trọng số khác nhau tùy thuộc vào độ chính xác tổng thể của chúng trên tập dữ liệu huấn luyện có trọng số.

CẢNH BÁO

Có một nhược điểm quan trọng của kỹ thuật học tuần tự này: quá trình huấn luyện không thể được thực hiện song song vì mỗi mô hình dự đoán chỉ có thể được huấn luyện sau khi mô hình dự đoán trước đó đã được huấn luyện và đánh giá. Do đó, nó không có khả năng mở rộng tốt như phương pháp bagging hoặc pasteing.

Hãy cùng xem xét kỹ hơn thuật toán AdaBoost. Trọng số của mỗi trường hợp w ban đầu được đặt là $1/m^{(1)}$. Một bộ dự đoán đầu tiên được huấn luyện, và tỷ lệ lỗi có trọng số r của nó được tính toán trên tập huấn luyện; xem [Phương trình 7-1](#).

Phương trình 7-1. Tỷ lệ lỗi có trọng số của biến dự đoán j

$$r_j = \frac{1}{m} \sum_{i=1}^m w^{(i)} |y^{(j)} - \hat{y}^{(i)}|$$
 trong đó $\hat{y}^{(i)}$ là dự đoán của bộ dự báo cho i^{th} ví dụ.

Trọng số α của bộ dự đoán sau đó được tính bằng [Phương trình 7-2](#), trong đó η là siêu tham số tốc độ học (mặc định là 1). Bộ dự đoán càng chính xác thì trọng số α của nó càng cao. Nếu nó chỉ đoán ngẫu nhiên, thì trọng số của nó sẽ gần bằng 0. Tuy nhiên, nếu nó thường xuyên đoán sai (tức là kém chính xác hơn so với đoán ngẫu nhiên), thì trọng số của nó sẽ âm.

Phương trình 7-2. Trọng số dự đoán

$$\alpha_j = \eta \log \frac{1}{r_j}$$

Tiếp theo, thuật toán AdaBoost cập nhật trọng số của các trường hợp bằng cách sử dụng [Phương trình 7-3](#), giúp tăng trọng số của các trường hợp bị phân loại sai.

Phương trình 7-3. Quy tắc cập nhật trọng lượng

với $i = 1, 2, \dots, m$

$$w^{(i)}_{(T+1)} = \begin{cases} w^{(i)}_{(T)} & \text{nếu } y^{(i)} = \hat{y}^{(i)} \\ w^{(i)}_{(T)} \exp(\alpha_j) & \text{nếu } y^{(i)} \neq \hat{y}^{(i)} \end{cases}$$

Sau đó, tất cả các trọng số của trường hợp được chuẩn hóa (tức là chia cho $\sum_{i=1}^m w^{(i)}$).

Cuối cùng, một mô hình dự đoán mới được huấn luyện bằng cách sử dụng các trọng số đã được cập nhật, và toàn bộ quá trình được lặp lại: trọng số của mô hình dự đoán mới được tính toán, trọng số của các trường hợp được cập nhật, sau đó một mô hình dự đoán khác được huấn luyện, và cứ thế tiếp diễn. Thuật toán dừng lại khi đạt được số lượng mô hình dự đoán mong muốn, hoặc khi tìm thấy một mô hình dự đoán hoàn hảo.

Để đưa ra dự đoán, AdaBoost chỉ đơn giản là tính toán các dự đoán của tất cả các biến dự đoán và gán trọng số cho chúng bằng cách sử dụng trọng số biến dự đoán α_j đa số. Lớp được dự đoán là lớp nhận được phiếu bầu có trọng số (xem **Phương trình 7-4**).

Phương trình 7-4. Dự đoán AdaBoost

$$\hat{y}(x) = \underset{k}{\operatorname{argmax}} \sum_{j=1}^N \alpha_j, \text{ trong đó } N \text{ là số lượng biến dự đoán.}$$

$\hat{y}_j(x) = k$

Scikit-Learn sử dụng phiên bản đa lớp của AdaBoost có tên là **SAMME**.¹⁶ (viết tắt của Stagewise Additive Modeling using a Multiclass Exponential loss function - Mô hình cộng dồn từng giai đoạn sử dụng hàm mất mát lũy thừa đa lớp). Khi chỉ có hai lớp, SAMME tương đương với AdaBoost. Nếu các mô hình dự đoán có thể ước tính xác suất của lớp (tức là nếu chúng có phương thức `predict_proba()`), Scikit-Learn có thể sử dụng một biến thể của SAMME gọi là SAMME.R (chữ R viết tắt của "Real" - Thực tế), dựa trên xác suất của lớp thay vì dự đoán và nhìn chung cho hiệu suất tốt hơn.

Đoạn mã sau huấn luyện một bộ phân loại AdaBoost dựa trên 30 Cây Quyết định (Decision Stump) bằng cách sử dụng lớp `AdaBoostClassifier` của Scikit-Learn (như bạn có thể đoán, cũng có lớp `AdaBoostRegressor`). Cây Quyết định là một Cây Quyết định với `max_depth=1` – nói cách khác, một cây bao gồm một nút quyết định duy nhất cộng với hai nút lá. Đây là bộ ước lượng cơ bản mặc định cho lớp `AdaBoostClassifier`:

```
from sklearn.ensemble import AdaBoostClassifier

ada_clf = AdaBoostClassifier(
    DecisionTreeClassifier(max_depth=1), n_estimators=30,
    learning_rate=0.5, random_state=42)
ada_clf.fit(X_train, y_train)
```

MẸO

Nếu mô hình AdaBoost của bạn bị quá khớp với tập dữ liệu huấn luyện, bạn có thể thử giảm số lượng mô hình ước lượng hoặc điều chỉnh mạnh hơn mô hình ước lượng cơ sở.

Tăng cường độ dốc

Một thuật toán tăng cường rất phổ biến khác là **Gradient Boosting**.¹⁷ Giống như AdaBoost, Gradient Boosting hoạt động bằng cách thêm tuần tự các mô hình dự đoán vào một tập hợp, mỗi mô hình sẽ hiệu chỉnh mô hình trước đó. Tuy nhiên, thay vì điều chỉnh trọng số của từng mô hình ở mỗi lần lặp như AdaBoost, phương pháp này cố gắng điều chỉnh mô hình dự đoán mới sao cho phù hợp với các lỗi còn lại do mô hình dự đoán trước đó tạo ra.

Hãy cùng xem xét một ví dụ hồi quy đơn giản, sử dụng Cây quyết định làm biến dự đoán cơ bản: đây được gọi là Tăng cường cây gradient, hay Cây hồi quy tăng cường gradient (GBRT).

Đầu tiên, hãy tạo một tập dữ liệu bậc hai nhiễu và huấn luyện mô hình DecisionTreeRegressor trên đó:

```
import numpy as np from
sklearn.tree import DecisionTreeRegressor

np.random.seed(42)
X = np.random.rand(100, 1) - 0.5 y = 3 *
X[:, 0] ** 2 + 0.05 * np.random.randn(100) # y = 3x2 + nhiễu Gaussian

tree_reg1 = DecisionTreeRegressor(max_depth=2, random_state=42) tree_reg1.fit(X,
y)
```

Tiếp theo, chúng ta sẽ huấn luyện một DecisionTreeRegressor thứ hai dựa trên các lỗi dư do bộ dự đoán đầu tiên tạo ra:

```
y2 = y - tree_reg1.predict(X)
tree_reg2 = DecisionTreeRegressor(max_depth=2, random_state=43) tree_reg2.fit(X,
y2)
```

Tiếp theo, chúng ta huấn luyện một mô hình hồi quy thứ ba dựa trên các sai số còn lại do mô hình dự đoán thứ hai tạo ra:

```
y3 = y2 - tree_reg2.predict(X)
tree_reg3 = DecisionTreeRegressor(max_depth=2, random_state=44) tree_reg3.fit(X,
y3)
```

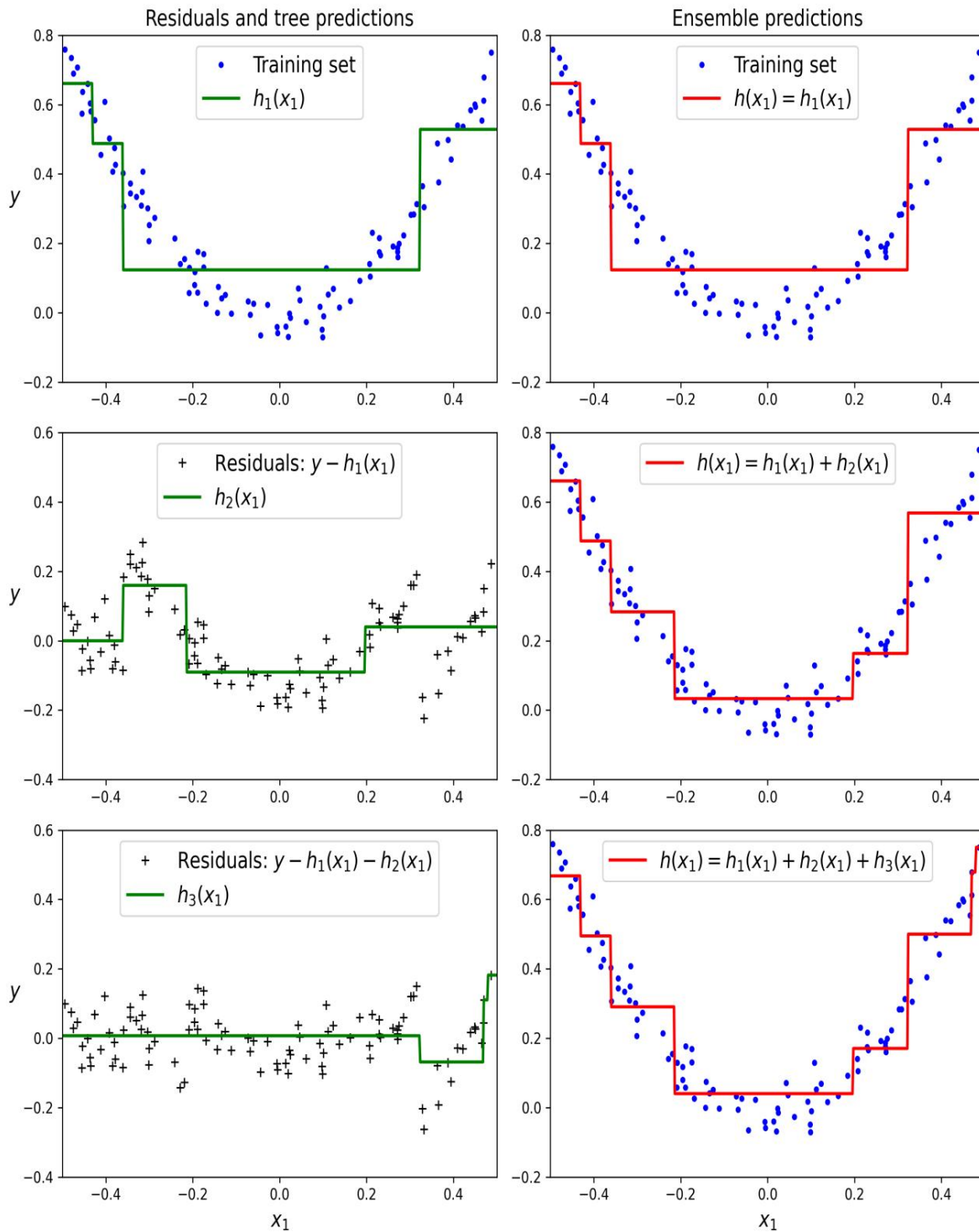
Giờ đây chúng ta có một mô hình kết hợp gồm ba cây quyết định. Mô hình này có thể đưa ra dự đoán về một trường hợp mới chỉ bằng cách cộng các dự đoán của tất cả các cây lại với nhau:

```
>>> X_new = np.array([[-0.4], [0.], [0.5]]) >>>
sum(tree.predict(X_new) for tree in (tree_reg1, tree_reg2, tree_reg3)) array([0.49484029,
0.04021166, 0.75026781])
```

Hình 7-9 thể hiện dự đoán của ba cây quyết định này ở cột bên trái và dự đoán của mô hình kết hợp ở cột bên phải. Ở hàng đầu tiên, mô hình kết hợp chỉ có một cây quyết định, vì vậy dự đoán của nó hoàn toàn giống với dự đoán của cây quyết định đầu tiên. Ở hàng thứ hai, một cây quyết định mới được huấn luyện dựa trên sai số dư của cây quyết định đầu tiên. Ở bên phải, bạn có thể thấy rằng dự đoán của mô hình kết hợp bằng tổng dự đoán của hai cây quyết định đầu tiên.

Tương tự, ở hàng thứ ba, một cây khác được huấn luyện dựa trên các lỗi còn lại của cây thứ hai.

Bạn có thể thấy rằng các dự đoán của mô hình kết hợp dần trở nên chính xác hơn khi số lượng cây quyết định được thêm vào mô hình.



Hình 7-9. Trong hình minh họa về Gradient Boosting này, bộ dự đoán đầu tiên (trên cùng bên trái) được huấn luyện bình thường, sau đó mỗi bộ dự đoán tiếp theo (giữa bên trái và dưới cùng bên trái) được huấn luyện trên phần dư của bộ dự đoán trước đó; cột bên phải hiển thị các dự đoán của tập hợp mô hình thu được.

Một cách đơn giản hơn để huấn luyện các mô hình GBRT là sử dụng lớp `GradientBoostingRegressor` của Scikit-Learn (cũng có lớp `GradientBoostingClassifier` dành cho phân loại). Tương tự như vậy...

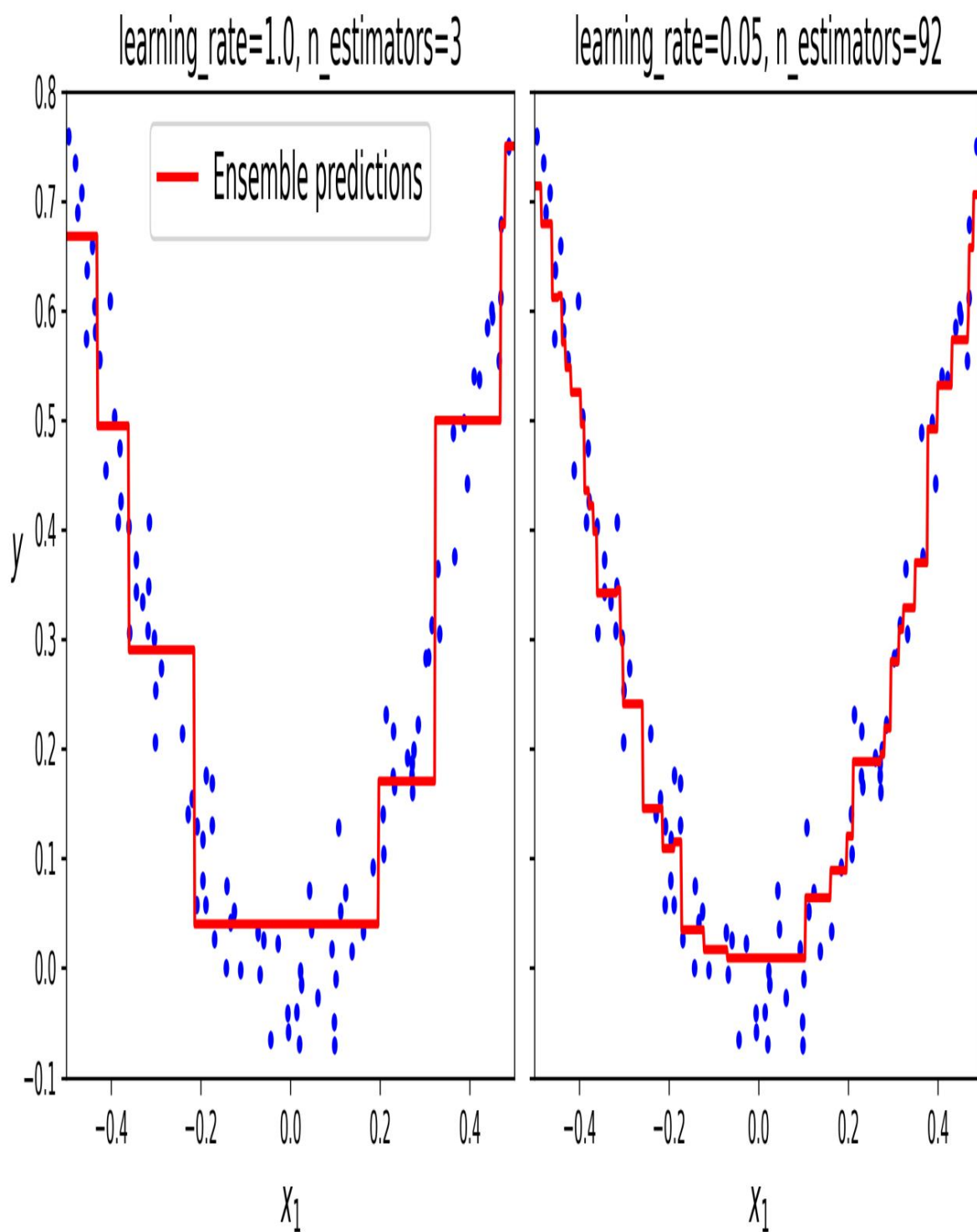
Lớp RandomForestRegressor có các siêu tham số để kiểm soát sự phát triển của Cây quyết định (ví dụ: max_depth, min_samples_leaf), cũng như các siêu tham số để kiểm soát quá trình huấn luyện ensemble, chẳng hạn như số lượng cây (n_estimators). Đoạn mã sau tạo ra cùng một ensemble như ensemble trước đó:

```
from sklearn.ensemble import GradientBoostingRegressor

gbrt = GradientBoostingRegressor(max_depth=2, n_estimators=3,
                                  learning_rate=1.0, random_state=42)

gbrt.fit(X, y)
```

Siêu tham số learning_rate điều chỉnh đóng góp của mỗi cây. Nếu bạn đặt nó ở giá trị thấp, chẳng hạn như 0,05, bạn sẽ cần nhiều cây hơn trong tập hợp để phù hợp với tập huấn luyện, nhưng các dự đoán thường sẽ tổng quát hóa tốt hơn. Đây là một kỹ thuật điều chỉnh được gọi là thu hẹp (shrinkage). Hình 7-10 cho thấy hai tập hợp GBRT được huấn luyện với các siêu tham số khác nhau: tập hợp bên trái không có đủ cây để phù hợp với tập huấn luyện, trong khi tập hợp bên phải có số lượng cây gần như phù hợp. Nếu chúng ta thêm nhiều cây hơn, GBRT sẽ bắt đầu quá khớp với tập huấn luyện.



Hình 7-10. Các mô hình GBRT với số lượng biến dự đoán không đủ (bên trái) và số lượng biến dự đoán vừa đủ (bên phải)

Để tìm số lượng cây tối ưu, bạn có thể thực hiện kiểm định chéo bằng cách sử dụng `GridSearchCV` hoặc `RandomizedSearchCV`, nhưng có một cách đơn giản hơn: nếu bạn đặt siêu tham số `n_iter_no_change` thành một giá trị nguyên, ví dụ như 10, thì... `GradientBoostingRegressor` sẽ tự động ngừng thêm cây quyết định trong quá trình thực hiện.

Quá trình huấn luyện sẽ dừng lại nếu nó nhận thấy 10 cây quyết định cuối cùng không mang lại kết quả.

Đây đơn giản là dừng sớm (đã được giới thiệu trong [Chương 4](#)), nhưng với một chút kiên nhẫn: nó chấp nhận việc không có tiến triển trong vài lần lặp trước khi dừng lại. Hãy huấn luyện mô hình ensemble bằng cách sử dụng dừng sớm:

```
gbrt_best = GradientBoostingRegressor( max_depth=2,
                                       learning_rate=0.05, n_estimators=500, n_iter_no_change=10, random_state=42)
gbrt_best.fit(X, y)
```

Nếu bạn đặt `n_iter_no_change` quá thấp, quá trình huấn luyện có thể dừng quá sớm và mô hình sẽ bị underfitting. Nhưng nếu bạn đặt nó quá cao, nó sẽ bị overfitting. Chúng tôi cũng đặt tốc độ học khá nhỏ và số lượng bộ ước lượng cao, nhưng số lượng bộ ước lượng thực tế trong tập hợp được huấn luyện lại thấp hơn nhiều, nhờ vào việc dừng sớm:

```
>>> gbrt_best.n_estimators_ 92
```

Khi tham số `n_iter_no_change` được thiết lập, phương thức `fit()` sẽ tự động chia tập huấn luyện thành một tập huấn luyện nhỏ hơn và một tập kiểm định: điều này cho phép nó đánh giá hiệu suất của mô hình mỗi khi thêm một cây mới. Kích thước của tập kiểm định được điều khiển bởi siêu tham số `validation_fraction`, mặc định là 10%. Siêu tham số `tol` xác định mức cải thiện hiệu suất tối đa vẫn được coi là không đáng kể. Giá trị mặc định là 0.0001.

Lớp `GradientBoostingRegressor` cũng hỗ trợ tham số `subsample`, chỉ định tỷ lệ các mẫu huấn luyện được sử dụng để huấn luyện mỗi cây. Ví dụ, nếu `subsample=0.25`, thì mỗi cây được huấn luyện trên 25% số mẫu huấn luyện, được chọn ngẫu nhiên. Như bạn có thể đoán được, kỹ thuật này đánh đổi độ lệch cao hơn để có phương sai thấp hơn. Nó cũng giúp tăng tốc độ huấn luyện đáng kể. Kỹ thuật này được gọi là Tăng cường Gradient Ngẫu nhiên (Stochastic Gradient Boosting).

Tăng cường độ dốc dựa trên biểu đồ tần số

Scikit-Learn cũng cung cấp một phương pháp triển khai GBRT khác, được tối ưu hóa cho các tập dữ liệu lớn: Tăng cường Gradient dựa trên Histogram (HGB). Phương pháp này hoạt động bằng cách phân nhóm các đặc trưng đầu vào, thay thế chúng bằng các số nguyên. Số lượng nhóm được điều khiển bởi siêu tham số `max_bins`, mặc định là 255 và không thể đặt cao hơn mức này. Việc phân nhóm có thể giảm đáng kể số lượng ngưỡng có thể mà thuật toán huấn luyện cần đánh giá. Hơn nữa, làm việc với các số nguyên cho phép sử dụng các cấu trúc dữ liệu nhanh hơn và tiết kiệm bộ nhớ hơn. Và cách xây dựng các nhóm loại bỏ nhu cầu sắp xếp các đặc trưng khi huấn luyện từng cây.

Do đó, cách triển khai này có độ phức tạp tính toán là $O(n \times m)$ thay vì $O(n \times m \times \log(m))$, trong đó m là số lượng mẫu huấn luyện và n là số lượng mẫu.

các đặc điểm. Trên thực tế, điều này có nghĩa là HGB có thể huấn luyện nhanh hơn hàng trăm lần so với GBRT thông thường trên các tập dữ liệu lớn. Tuy nhiên, việc phân nhóm gây ra sự mất mát về độ chính xác, hoạt động như một yếu tố điều chỉnh: tùy thuộc vào tập dữ liệu, điều này có thể giúp giảm thiểu hiện tượng quá khớp hoặc có thể gây ra hiện tượng thiếu khớp.

Scikit-Learn cung cấp hai lớp cho HGB: `HistGradientBoostingRegressor` và `HistGradientBoostingClassifier`.

Chúng có API tương tự như `GradientBoostingRegressor` và `GradientBoostingClassifier`, với một

vài điểm khác biệt đáng chú ý:

- Chức năng dừng sớm sẽ tự động được kích hoạt nếu số lượng phiên bản lớn hơn 10.000. Bạn có thể bật hoặc tắt chức năng dừng sớm bằng cách đặt siêu tham số `early_stopping` thành `True` hoặc `False`.
- Tính năng lấy mẫu con không được hỗ trợ.
- `n_estimators` được đổi tên thành `max_iter`.
- Các siêu tham số của Cây quyết định duy nhất có thể được điều chỉnh là `max_leaf_nodes`, `min_samples_leaf` và `max_depth`.

Các lớp HGB cũng có hai tính năng tuyệt vời: chúng hỗ trợ cả các đặc trưng phân loại và giá trị thiếu.

Điều này giúp đơn giản hóa quá trình tiền xử lý khá nhiều. Tuy nhiên, các đặc trưng phân loại phải được biểu diễn dưới dạng số nguyên nằm trong khoảng từ 0 đến một số nhỏ hơn `max_bins`: bạn có thể sử dụng

`OrdinalEncoder` cho việc này. Ví dụ, đây là cách xây dựng và huấn luyện một quy trình hoàn chỉnh cho

tập dữ liệu nhà ở California được giới thiệu trong [Chương 2](#):

```
from sklearn.pipeline import make_pipeline
from sklearn.compose import make_column_transformer
from sklearn.ensemble import HistGradientBoostingRegressor
from sklearn.preprocessing import OrdinalEncoder

hgb_reg = make_pipeline(
    make_column_transformer((OrdinalEncoder(), ["ocean_proximity"]),
                           remainder="passthrough"),
    HistGradientBoostingRegressor(categorical_features=[0], random_state=42)
) hgb_reg.fit(housing, housing_labels)
```

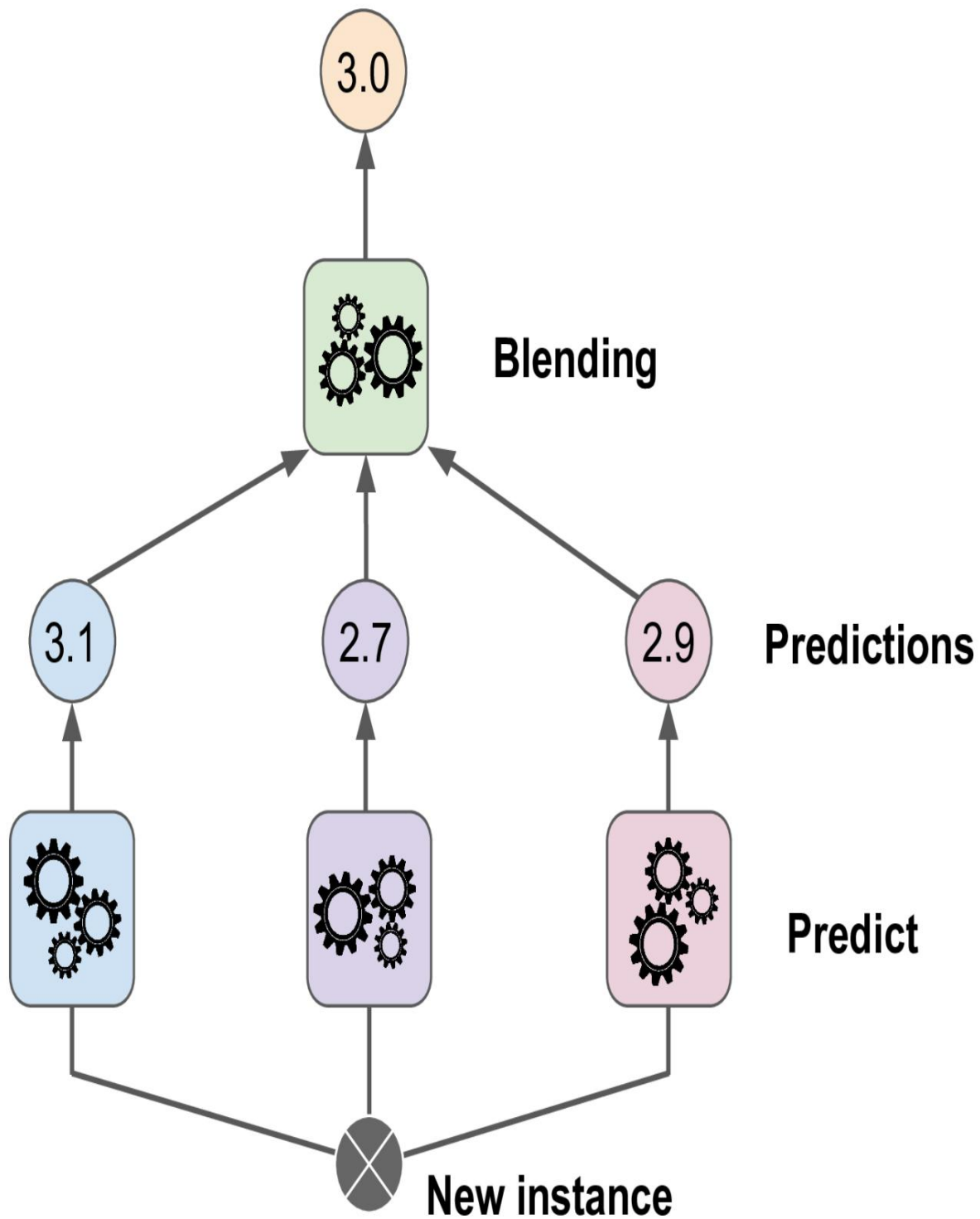
Toàn bộ quy trình xử lý dữ liệu ngắn gọn như chính các bước nhập liệu! Không cần đến bộ xử lý dữ liệu thiếu, bộ điều chỉnh tỷ lệ hay bộ mã hóa one-hot, thực sự rất tiện lợi. Lưu ý rằng `categorical_features` phải được đặt thành chỉ số cột phân loại (hoặc một mảng boolean). Không cần tinh chỉnh siêu tham số, mô hình này cho ra RMSE khoảng 47.600, không quá tệ.

MẸO

Một số triển khai tối ưu hóa khác của Gradient Boosting cũng có sẵn trong hệ sinh thái Python ML, cụ thể là: **XGBoost**, **CatBoost**, và **LightGBM**. Các thư viện này đã tồn tại nhiều năm, tất cả đều chuyên về Gradient Boosting, API của chúng rất giống với Scikit-Learn và cung cấp nhiều tính năng bổ sung, bao gồm cả tăng tốc GPU: bạn chắc chắn nên xem xét chúng! Hơn nữa, còn có **thư viện TensorFlow Random Forests**. Cung cấp các triển khai tối ưu hóa của nhiều thuật toán Rừng ngẫu nhiên: Rừng ngẫu nhiên thông thường, Cây bổ sung, GBRT và một số thuật toán khác.

Xếp chồng

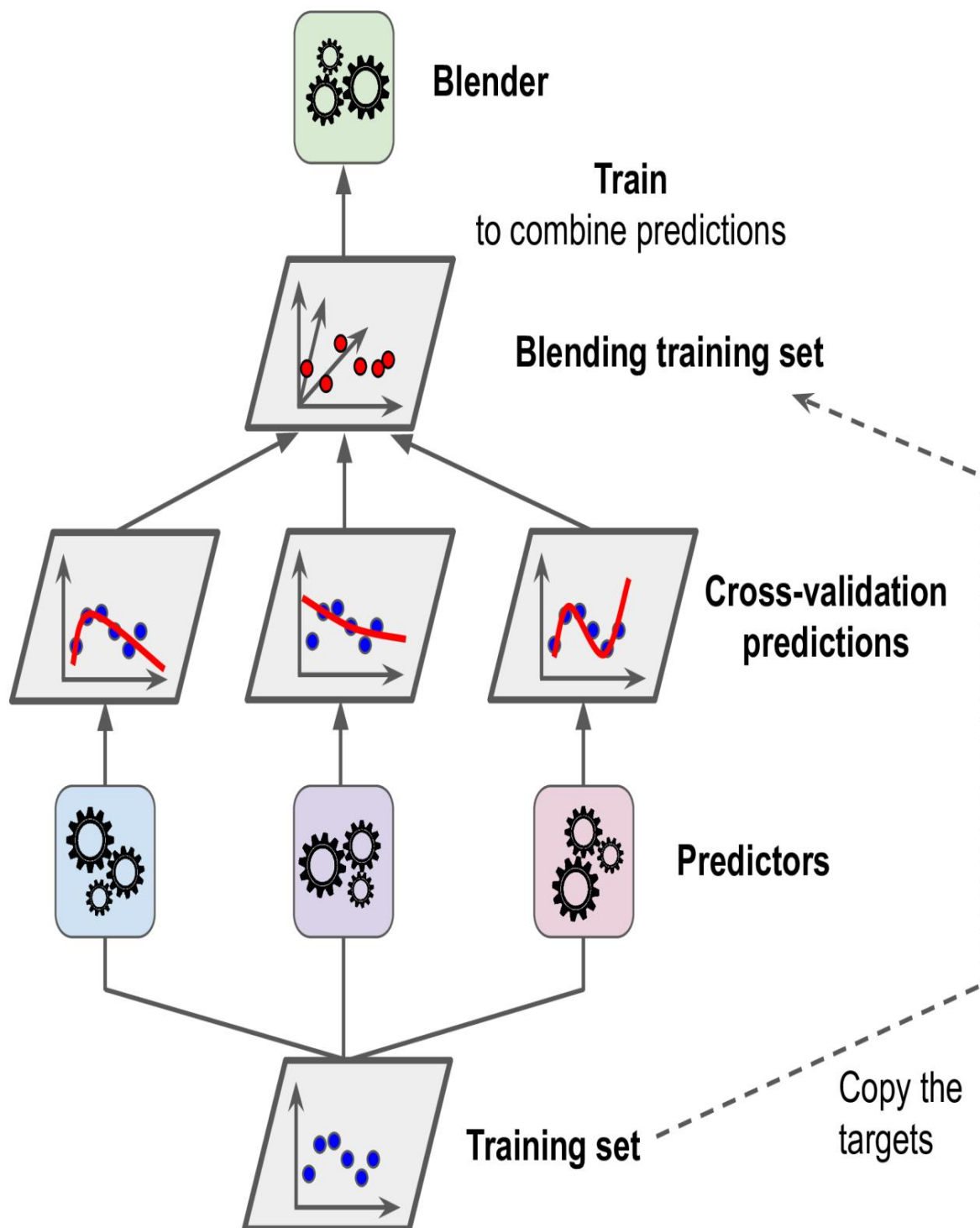
Phương pháp Ensemble cuối cùng mà chúng ta sẽ thảo luận trong chương này được gọi là xếp chồng (viết tắt của **stacked generalization**). Nó dựa trên một ý tưởng đơn giản: thay vì sử dụng các hàm tầm thường (như bỏ phiếu cứng) để tổng hợp các dự đoán của tất cả các mô hình dự đoán trong một tập hợp, tại sao chúng ta không huấn luyện một mô hình để thực hiện việc tổng hợp này? **Hình 7-11** cho thấy một tập hợp như vậy đang thực hiện nhiệm vụ hồi quy trên một trường hợp mới. Mỗi trong ba mô hình dự đoán phía dưới dự đoán một giá trị khác nhau (3.1, 2.7 và 2.9), và sau đó mô hình dự đoán cuối cùng (được gọi là bộ trộn, hoặc bộ học siêu cấp) lấy các dự đoán này làm đầu vào và đưa ra dự đoán cuối cùng (3.0).



Hình 7-11. Tổng hợp các dự đoán bằng cách sử dụng bộ dự đoán kết hợp.

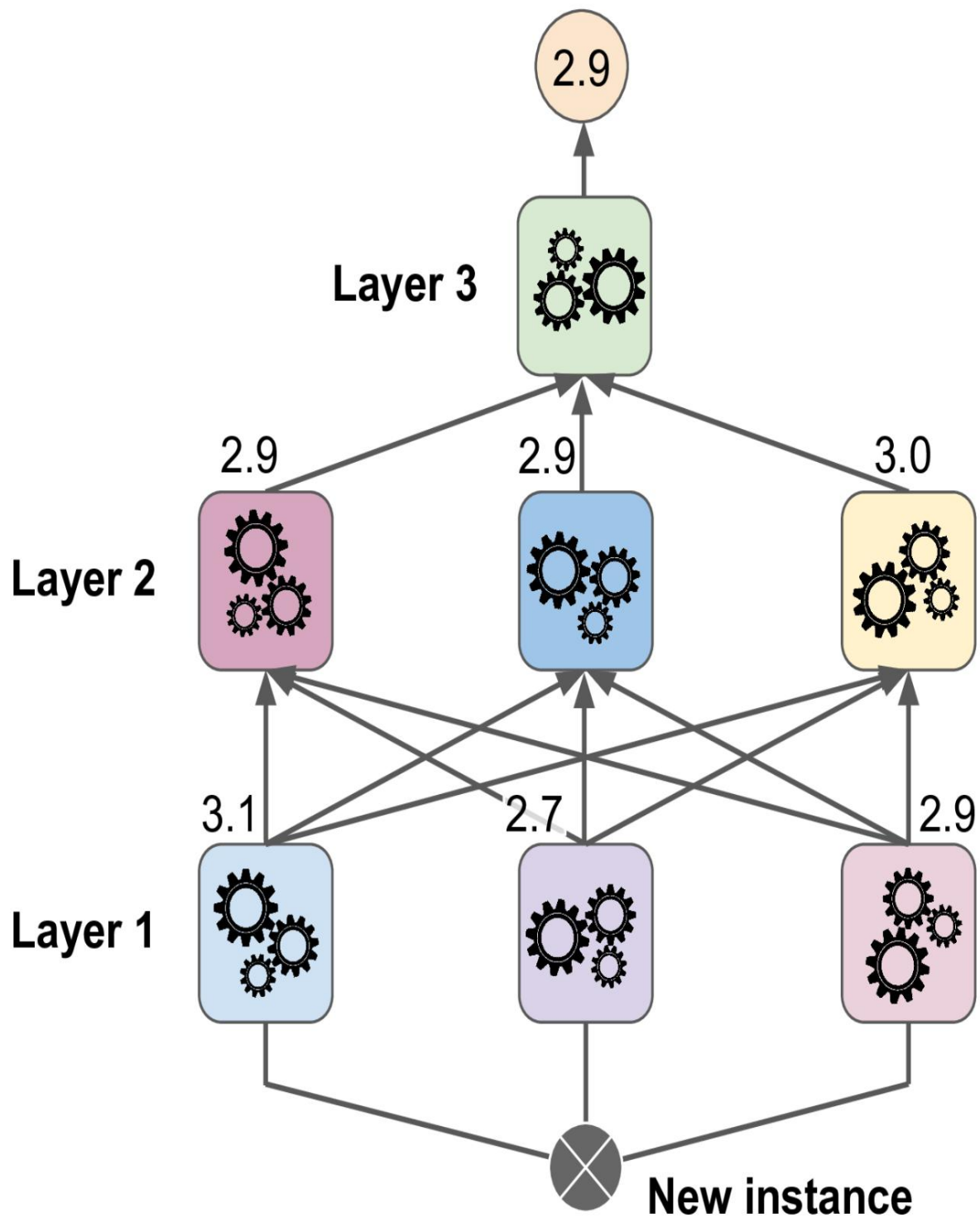
Để huấn luyện bộ trộn, trước tiên bạn cần xây dựng tập dữ liệu huấn luyện trộn: bạn có thể sử dụng `cross_val_predict()` trên mỗi bộ dự đoán trong tập hợp để nhận được các dự đoán ngoài mẫu cho mỗi trường hợp trong tập dữ liệu huấn luyện gốc (Hình 7-12). Những dự đoán này có thể được sử dụng làm các đặc trưng đầu vào để huấn luyện bộ trộn, và các mục tiêu có thể được sao chép đơn giản từ đó.

Tập dữ liệu huấn luyện gốc. Lưu ý rằng bất kể số lượng đặc trưng trong tập dữ liệu huấn luyện gốc (chỉ một trong ví dụ này), tập dữ liệu huấn luyện kết hợp sẽ chứa một đặc trưng đầu vào cho mỗi bộ dự đoán (ba trong ví dụ này). Sau khi bộ kết hợp được huấn luyện, các bộ dự đoán cơ sở được huấn luyện lại lần cuối trên toàn bộ tập dữ liệu huấn luyện gốc.



Hình 7-12. Huấn luyện bộ trộn trong một tập hợp xếp chồng.

Thực tế, hoàn toàn có thể huấn luyện nhiều bộ trộn khác nhau theo cách này (ví dụ: một bộ sử dụng Hồi quy tuyến tính, một bộ khác sử dụng Hồi quy rừng ngẫu nhiên), để tạo ra một lớp bộ trộn hoàn chỉnh, và sau đó thêm một bộ trộn khác lên trên để tạo ra dự đoán cuối cùng, như minh họa trong **Hình 7-13**. Bạn có thể cải thiện hiệu suất thêm một chút bằng cách này, nhưng sẽ phải trả giá bằng thời gian huấn luyện và độ phức tạp của hệ thống.



Hình 7-13. Dự đoán trong một tập hợp xếp chồng nhiều lớp.

Scikit-Learn cung cấp hai lớp để kết hợp các mô hình: `StackingClassifier` và `StackingRegressor`. Ví dụ, bạn có thể thay thế `VotingClassifier` mà bạn đã sử dụng ở đầu chương này trên tập dữ liệu moons bằng một `StackingClassifier`:

```

from sklearn.ensemble import StackingClassifier

stacking_clf =
    StackingClassifier( estimators=[
        ('lr', LogisticRegression(random_state=42)), ('rf',
        RandomForestClassifier(random_state=42)), ('svc',
        SVC(probability=True, random_state=42))
    ],
    final_estimator=RandomForestClassifier(random_state=43), cv=5 # số lần lặp kiểm định chéo

) stacking_clf.fit(X_train, y_train)

```

Đối với mỗi biến dự đoán, bộ phân loại xếp chồng sẽ gọi hàm `predict_proba()` nếu có sẵn, hoặc sẽ sử dụng hàm `decision_function()` nếu có sẵn, hoặc trong trường hợp bất khả kháng, nó sẽ gọi hàm `predict()`. Nếu bạn không cung cấp bộ ước lượng cuối cùng, `StackingClassifier` sẽ sử dụng `LogisticRegression`, và `StackingRegressor` sẽ sử dụng `RidgeCV`.

Nếu bạn đánh giá mô hình xếp chồng này trên tập dữ liệu thử nghiệm, bạn sẽ thấy độ chính xác là 92,8%, tốt hơn một chút so với bộ phân loại bỏ phiếu sử dụng phương pháp bỏ phiếu mềm, đạt 92%.

Tóm lại, các phương pháp kết hợp (ensemble methods) rất linh hoạt, mạnh mẽ và khá dễ sử dụng. Rừng ngẫu nhiên (Random Forests), AdaBoost và GBRT nằm trong số những mô hình đầu tiên bạn nên thử nghiệm trên hầu hết các tác vụ Học máy, và chúng đặc biệt hiệu quả với dữ liệu dạng bảng không đồng nhất. Hơn nữa, vì chúng yêu cầu rất ít bước tiền xử lý, chúng rất tuyệt vời để nhanh chóng xây dựng và vận hành một nguyên mẫu. Cuối cùng, các phương pháp kết hợp như bộ phân loại bỏ phiếu (voting classifiers) và bộ phân loại xếp chồng (stacking classifiers) có thể giúp đẩy hiệu năng của hệ thống lên mức tối đa.

Bài tập

1. Nếu bạn đã huấn luyện năm mô hình khác nhau trên cùng một tập dữ liệu huấn luyện và tất cả đều đạt độ chính xác 95%, liệu có khả năng nào bạn có thể kết hợp các mô hình này để đạt được kết quả tốt hơn không? Nếu có, bằng cách nào? Nếu không, tại sao?
2. Sự khác biệt giữa bộ phân loại bỏ phiếu cứng và bộ phân loại bỏ phiếu mềm là gì?
3. Liệu có thể tăng tốc quá trình huấn luyện một mô hình kết hợp bagging bằng cách phân tán nó trên nhiều máy chủ không? Còn các mô hình kết hợp pasteing, boosting, Random Forests, hay stacking thì sao?
4. Lợi ích của việc đánh giá ngoài tập dữ liệu huấn luyện là gì?
5. Điều gì khiến Extra-Trees ngẫu nhiên hơn so với Random Forests thông thường? Tính ngẫu nhiên bổ sung này có thể giúp ích như thế nào? Extra-Trees chậm hơn hay nhanh hơn so với Random Forests thông thường?

6. Nếu mô hình AdaBoost của bạn không khớp với dữ liệu huấn luyện, bạn nên điều chỉnh những siêu tham số nào và điều chỉnh như thế nào?
7. Nếu mô hình Gradient Boosting của bạn bị quá khớp với tập dữ liệu huấn luyện, bạn nên tăng hay giảm tốc độ học?
8. Tải dữ liệu MNIST (đã giới thiệu trong [Chương 3](#)) và chia nó thành tập huấn luyện, tập xác thực và tập kiểm tra (ví dụ: sử dụng 50.000 mẫu để huấn luyện, 10.000 mẫu để xác thực và 10.000 mẫu để kiểm tra). Sau đó, huấn luyện các bộ phân loại khác nhau, chẳng hạn như bộ phân loại Rừng ngẫu nhiên, bộ phân loại Cây bổ sung và bộ phân loại SVM. Tiếp theo, hãy thử kết hợp chúng thành một mô hình tổng hợp có hiệu suất vượt trội hơn từng bộ phân loại riêng lẻ trên tập xác thực, sử dụng phương pháp bỏ phiếu mềm hoặc bỏ phiếu cứng. Khi đã tìm được một mô hình phù hợp, hãy thử nghiệm nó trên tập kiểm tra. Hiệu suất của nó tốt hơn bao nhiêu so với các bộ phân loại riêng lẻ?
9. Chạy các bộ phân loại riêng lẻ từ bài tập trước để đưa ra dự đoán trên tập dữ liệu xác thực, và tạo một tập dữ liệu huấn luyện mới với các dự đoán thu được: mỗi trường hợp huấn luyện là một vectơ chứa tập hợp các dự đoán từ tất cả các bộ phân loại của bạn cho một hình ảnh, và mục tiêu là lớp của hình ảnh đó. Huấn luyện một bộ phân loại trên tập dữ liệu huấn luyện mới này. Chúc mừng, bạn vừa huấn luyện một bộ trộn (blender), và cùng với các bộ phân loại, nó tạo thành một tập hợp xếp chồng (stacking ensemble)! Bây giờ hãy đánh giá tập hợp này trên tập dữ liệu kiểm tra. Đối với mỗi hình ảnh trong tập dữ liệu kiểm tra, hãy đưa ra dự đoán bằng tất cả các bộ phân loại của bạn, sau đó đưa các dự đoán đó vào bộ trộn để nhận được dự đoán của tập hợp. Kết quả so sánh như thế nào với bộ phân loại bỏ phiếu mà bạn đã huấn luyện trước đó? Bây giờ hãy thử lại bằng cách sử dụng StackingClassifier: bạn có nhận được hiệu suất tốt hơn không? Nếu có, tại sao?

Đáp án cho các bài tập này có sẵn ở cuối tập tài liệu của chương này, tại địa chỉ <https://hom1.info/colab3>.

1. Hãy tưởng tượng bạn chọn ngẫu nhiên một lá bài từ bộ bài, viết nó xuống, rồi đặt lại vào bộ bài trước khi...
Chọn lá bài tiếp theo: cùng một lá bài có thể được chọn nhiều lần.

2 Leo Breiman, "Bagging Predictors," Machine Learning 24, no. 2 (1996): 123-140.

3. Trong thống kê, việc lấy mẫu lại có hoàn trả được gọi là bootstrapping.

4 Leo Breiman, "Dán các phiếu bầu nhỏ để phân loại trong cơ sở dữ liệu lớn và trực tuyến," Học máy 36, số 1-2 (1999): 85-103.

5. Khái niệm về độ lệch và phương sai đã được giới thiệu trong [Chương 4](#).

Tham số max_samples có thể được đặt thành một số thực nằm giữa 0,0 và 1,0, trong trường hợp đó số lượng mẫu tối đa được lấy là 6.
instances bằng kích thước của tập huấn luyện nhân với max_samples.

7 Khi m tăng lên, tỷ lệ này tiến tới $1 - \exp(-1) \approx 63\%$.

8 Gilles Louppe và Pierre Geurts, "Các tập hợp trên các mảng ngẫu nhiên," Ghi chú bài giảng về Khoa học máy tính 7523 (2012): 346-361.

9 Tin Kam Ho, "Phương pháp không gian con ngẫu nhiên để xây dựng rừng quyết định," Giao dịch IEEE về mô hình Phân tích và Trí tuệ Máy móc 20, số 8 (1998): 832-844.

10. Tin Kam Ho, "Rừng quyết định ngẫu nhiên", Kỷ yếu Hội nghị quốc tế lần thứ ba về Tài liệu Phân tích và Nhận dạng 1 (1995): 278.
11. Lớp BaggingClassifier vẫn hữu ích nếu bạn muốn tạo một tập hợp các đối tượng khác ngoài Cây quyết định.
12. Pierre Geurts et al., "Cây ngẫu nhiên cực độ," Học máy 63, số 1 (2006): 3-42.
13. Yoav Freund và Robert E. Schapire, "Một sự khái quát hóa lý thuyết quyết định về học trực tuyến và ứng dụng vào Boosting," Tạp chí Khoa học Máy tính và Hệ thống 55, số 1 (1997): 119-139.
14. Điều này chỉ mang tính chất minh họa. Nhìn chung, SVM không phải là mô hình dự đoán cơ bản tốt cho AdaBoost; chúng hoạt động chậm. và có xu hướng không ổn định với điều đó.
15. Thuật toán AdaBoost gốc không sử dụng siêu tham số tốc độ học.
16. Để biết thêm chi tiết, xem Ji Zhu et al., "Multi-Class AdaBoost," Statistics and Its Interface 2, no. 3 (2009): 349-360.
17. Phương pháp Gradient Boosting lần đầu tiên được giới thiệu trong bài báo "Arcing the Edge" năm 1997 của Leo Breiman. và còn hơn thế nữa được phát triển trong bài báo năm 1999 "Phương pháp xấp xỉ hàm tham lam: Một cỗ máy tăng cường độ dốc" của Jerome H. Friedman.
18. David H. Wolpert, "Tổng quát hóa xếp chồng," Mạng thần kinh 5, số 2 (1992): 241-259.

oceanofPDE.com

Chương 8. Giảm chiều dữ liệu

Với sách điện tử phát hành sớm, bạn sẽ nhận được sách ở dạng sơ khai nhất—nội dung thô và chưa chỉnh sửa của tác giả khi họ viết—nhờ đó bạn có thể tận dụng những công nghệ này rất lâu trước khi các tựa sách được phát hành chính thức.

Đây sẽ là chương thứ 8 của cuốn sách cuối cùng. Kho lưu trữ GitHub là <https://github.com/ageron/handson-ml3>.

Nếu bạn có ý kiến đóng góp về cách chúng tôi có thể cải thiện nội dung và/hoặc ví dụ trong cuốn sách này, hoặc nếu bạn nhận thấy thiếu sót nào trong chương này, vui lòng liên hệ với biên tập viên theo địa chỉ mcronin@oreilly.com.

Nhiều bài toán Học máy liên quan đến hàng nghìn, thậm chí hàng triệu đặc trưng cho mỗi trường hợp huấn luyện. Không chỉ tất cả các đặc trưng này làm cho quá trình huấn luyện cực kỳ chậm, mà chúng còn có thể khiến việc tìm ra một giải pháp tốt trở nên khó khăn hơn nhiều, như chúng ta sẽ thấy. Vấn đề này thường được gọi là lời nguyền về chiều không gian.

May mắn thay, trong các bài toán thực tế, thường có thể giảm đáng kể số lượng đặc trưng, biến một bài toán khó giải thành một bài toán khả thi. Ví dụ, hãy xem xét các ảnh MNIST (đã được giới thiệu trong [Chương 3](#)): các pixel ở viền ảnh hầu như luôn có màu trắng, vì vậy bạn có thể loại bỏ hoàn toàn các pixel này khỏi tập huấn luyện mà không làm mất nhiều thông tin. Như chúng ta đã thấy trong chương trước, [Hình 7-6](#) xác nhận rằng các pixel này hoàn toàn không quan trọng đối với nhiệm vụ phân loại.

Ngoài ra, hai điểm ảnh liền kề thường có mối tương quan cao: nếu bạn hợp nhất chúng thành một điểm ảnh duy nhất (ví dụ, bằng cách lấy giá trị trung bình của cường độ hai điểm ảnh), bạn sẽ không mất nhiều thông tin.

CẢNH BÁO

Việc giảm chiều dữ liệu sẽ gây ra một số mất mát thông tin, giống như việc nén ảnh thành JPEG có thể làm giảm chất lượng của ảnh, vì vậy mặc dù nó sẽ giúp tăng tốc quá trình huấn luyện, nhưng nó có thể làm cho hiệu suất của hệ thống giảm đi một chút. Nó cũng làm cho các pipeline của bạn phức tạp hơn và do đó khó bảo trì hơn. Vì vậy, tôi khuyên bạn nên thử huấn luyện hệ thống của mình với dữ liệu gốc trước khi xem xét sử dụng phương pháp giảm chiều dữ liệu. Trong một số trường hợp, việc giảm chiều dữ liệu huấn luyện có thể lọc bỏ một số nhiễu và chi tiết không cần thiết và do đó dẫn đến hiệu suất cao hơn, nhưng nói chung thì không; nó chỉ giúp tăng tốc quá trình huấn luyện mà thôi.

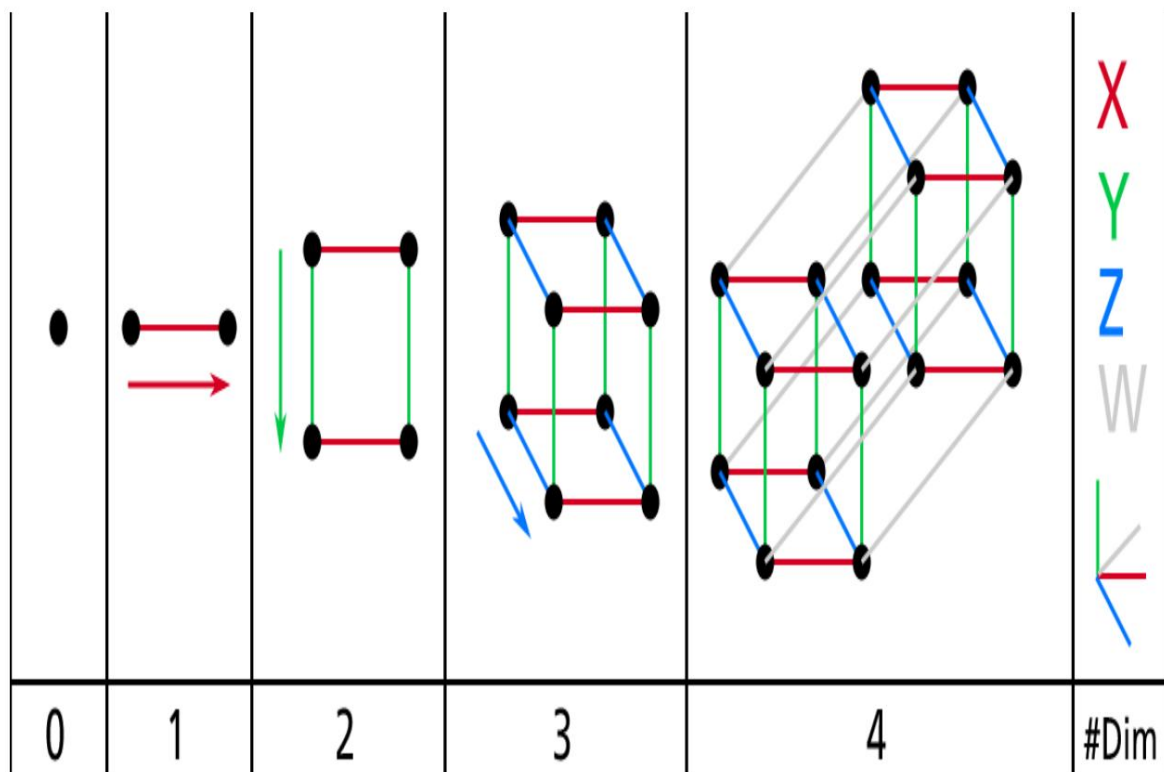
Ngoài việc tăng tốc quá trình huấn luyện, giảm chiều dữ liệu còn cực kỳ hữu ích cho việc trực quan hóa dữ liệu (hay DataViz). Giảm số chiều xuống còn hai (hoặc ba) giúp có thể vẽ một cái nhìn cô đọng về tập dữ liệu huấn luyện đa chiều trên biểu đồ và thường thu được một số hiểu biết quan trọng bằng cách phát hiện trực quan các mẫu, chẳng hạn như các cụm.

Hơn nữa, trực quan hóa dữ liệu (DataViz) rất cần thiết để truyền đạt kết luận của bạn cho những người không phải là nhà khoa học dữ liệu – đặc biệt là những người ra quyết định sẽ sử dụng kết quả của bạn.

Trong chương này, chúng ta sẽ thảo luận về lời nguyên của chiều không gian và hiểu được những gì diễn ra trong không gian đa chiều. Sau đó, chúng ta sẽ xem xét hai phương pháp chính để giảm chiều không gian (phép chiếu và học đa tạp), và chúng ta sẽ tìm hiểu ba kỹ thuật giảm chiều không gian phổ biến nhất: PCA, phép chiếu ngẫu nhiên và LLE.

Lời nguyên của chiều không gian

Chúng ta đã quá quen với việc sống trong không gian ba chiều¹ đến nỗi trực giác của chúng ta không còn hoạt động hiệu quả khi cố gắng hình dung một không gian đa chiều. Ngay cả một hình lập phương siêu chiều 4D cơ bản cũng vô cùng khó hình dung trong tâm trí chúng ta (xem [Hình 8-1](#)), chứ chưa nói đến một hình elip 200 chiều bị uốn cong trong không gian 1.000 chiều.



Hình 8-1. Điểm, đoạn thẳng, hình vuông, hình lập phương và hình lập phương bốn chiều (hình lập phương siêu chiều từ 0D đến 4D)

Hóa ra, nhiều thứ hoạt động rất khác nhau trong không gian đa chiều. Ví dụ, nếu bạn chọn một điểm ngẫu nhiên trong một hình vuông đơn vị (hình vuông 1×1), nó chỉ có khoảng 0,4% khả năng nằm cách biên giới ít hơn 0,001 (nói cách khác, rất khó có khả năng một điểm ngẫu nhiên sẽ nằm ở vị trí "cực đoan" dọc theo bất kỳ chiều nào). Nhưng trong một siêu lập phương đơn vị 10.000 chiều, xác suất này lớn hơn 99,999999%. Hầu hết các điểm trong siêu lập phương đa chiều đều rất gần biên giới.

Đây là một sự khác biệt rắc rối hơn: nếu bạn chọn ngẫu nhiên hai điểm trong một hình vuông đơn vị, khoảng cách giữa hai điểm này trung bình sẽ là khoảng 0,52. Nếu bạn chọn ngẫu nhiên hai điểm trong một khối lập phương 3D đơn vị, khoảng cách trung bình sẽ là khoảng 0,66. Nhưng còn hai điểm được chọn ngẫu nhiên trong một siêu khối 1.000.000 chiều thì sao? Khoảng cách trung bình, dù bạn có tin hay không, sẽ là khoảng 408,25 (xấp xỉ $\sqrt{1.000.000/6}$)! Điều này trái với trực giác: làm thế nào hai điểm lại cách xa nhau như vậy khi cả hai đều nằm trong cùng một siêu khối đơn vị? Chà, đơn giản là có rất nhiều không gian trong không gian nhiều chiều. Kết quả là, các tập dữ liệu nhiều chiều có nguy cơ rất thưa thớt: hầu hết các trường hợp huấn luyện có khả năng nằm cách xa nhau. Điều này cũng có nghĩa là

Điều đó có nghĩa là một trường hợp mới có thể sẽ rất khác so với bất kỳ trường hợp huấn luyện nào, khiến cho các dự đoán kém tin cậy hơn nhiều so với trong không gian ít chiều hơn, vì chúng sẽ dựa trên các phép ngoại suy lớn hơn nhiều. Nói tóm lại, tập dữ liệu huấn luyện càng có nhiều chiều, nguy cơ quá khớp càng cao.

Về lý thuyết, một giải pháp cho vấn đề "lời nguyền về chiều không gian" có thể là tăng kích thước tập dữ liệu huấn luyện để đạt được mật độ mẫu huấn luyện đủ lớn.

Thật không may, trên thực tế, số lượng mẫu huấn luyện cần thiết để đạt được mật độ nhất định tăng theo cấp số mũ với số chiều.

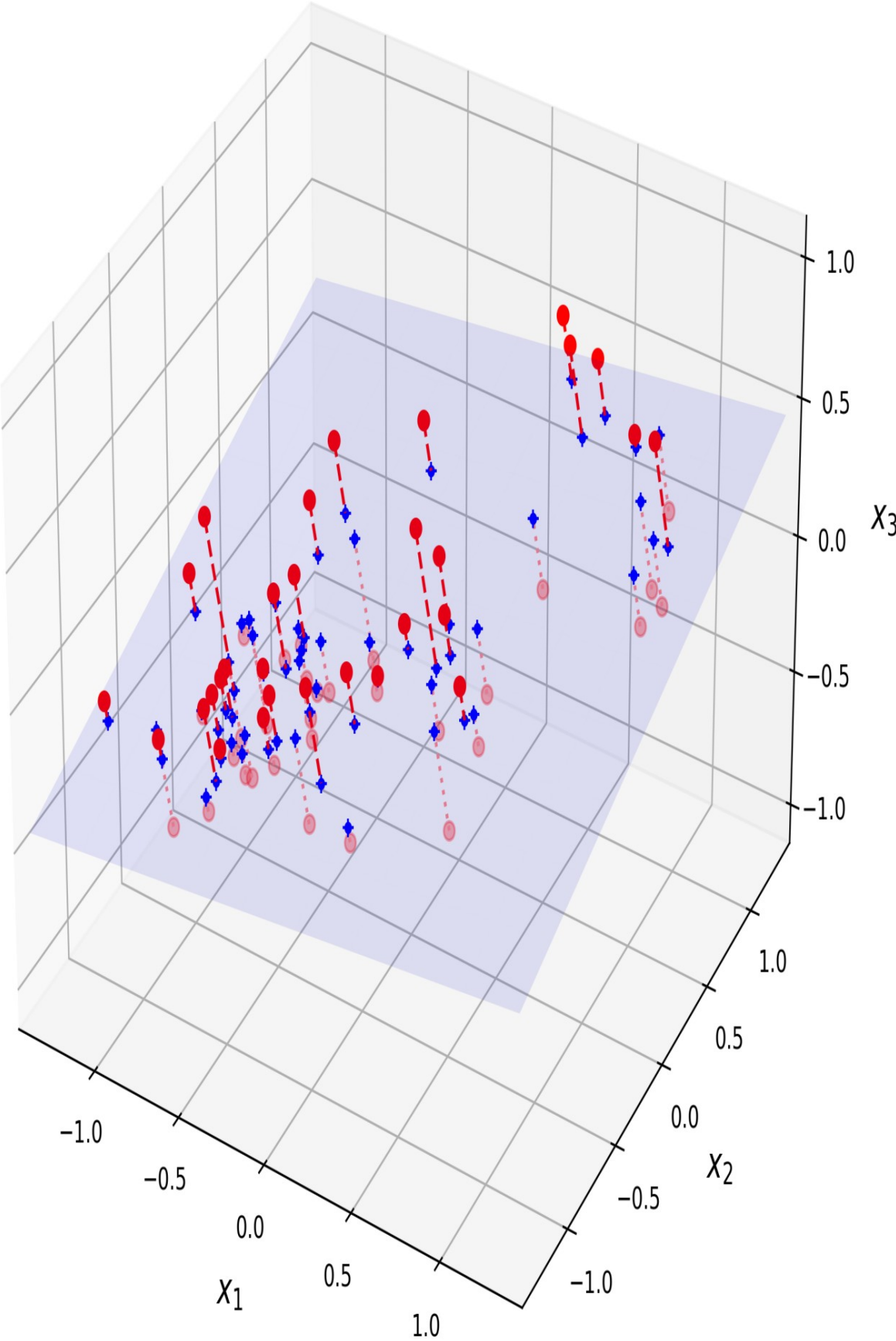
Với chỉ 100 đặc trưng—ít hơn đáng kể so với bài toán MNIST—tất cả đều nằm trong khoảng từ 0 đến 1, bạn sẽ cần nhiều mẫu huấn luyện hơn số nguyên tử trong vũ trụ quan sát được để các mẫu huấn luyện có giá trị trung bình chệch lệch nhau không quá 0,1, giả sử chúng được phân bố đồng đều trên tất cả các chiều.

Các phương pháp chính để tính chiều Sự giảm bớt

Trước khi đi sâu vào các thuật toán giảm chiều cụ thể, chúng ta hãy cùng xem xét hai phương pháp chính để giảm chiều: phép chiếu và học đa tạp.

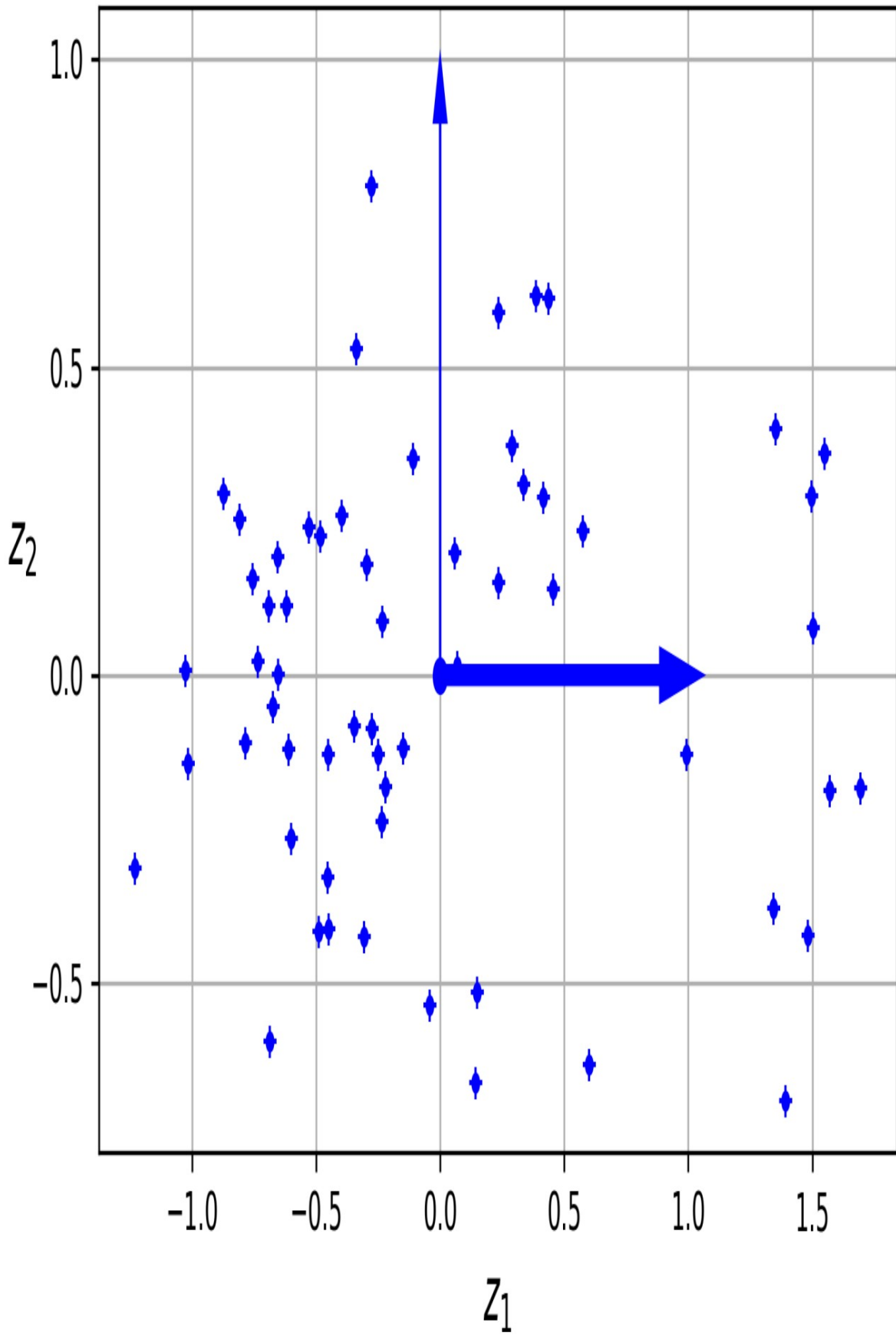
Phép chiếu

Trong hầu hết các bài toán thực tế, các mẫu huấn luyện không được phân bố đồng đều trên tất cả các chiều. Nhiều đặc trưng gần như không đổi, trong khi những đặc trưng khác lại có mối tương quan cao (như đã thảo luận trước đó với tập dữ liệu MNIST). Kết quả là, tất cả các mẫu huấn luyện nằm trong (hoặc gần) một không gian con có chiều thấp hơn nhiều so với không gian đa chiều. Điều này nghe có vẻ rất trừu tượng, vì vậy hãy xem xét một ví dụ. Trong **Hình 8-2**, bạn có thể thấy một tập dữ liệu 3D được biểu diễn bằng các hình cầu nhỏ.



Hình 8-2. Tập dữ liệu 3D nằm gần không gian con 2D.

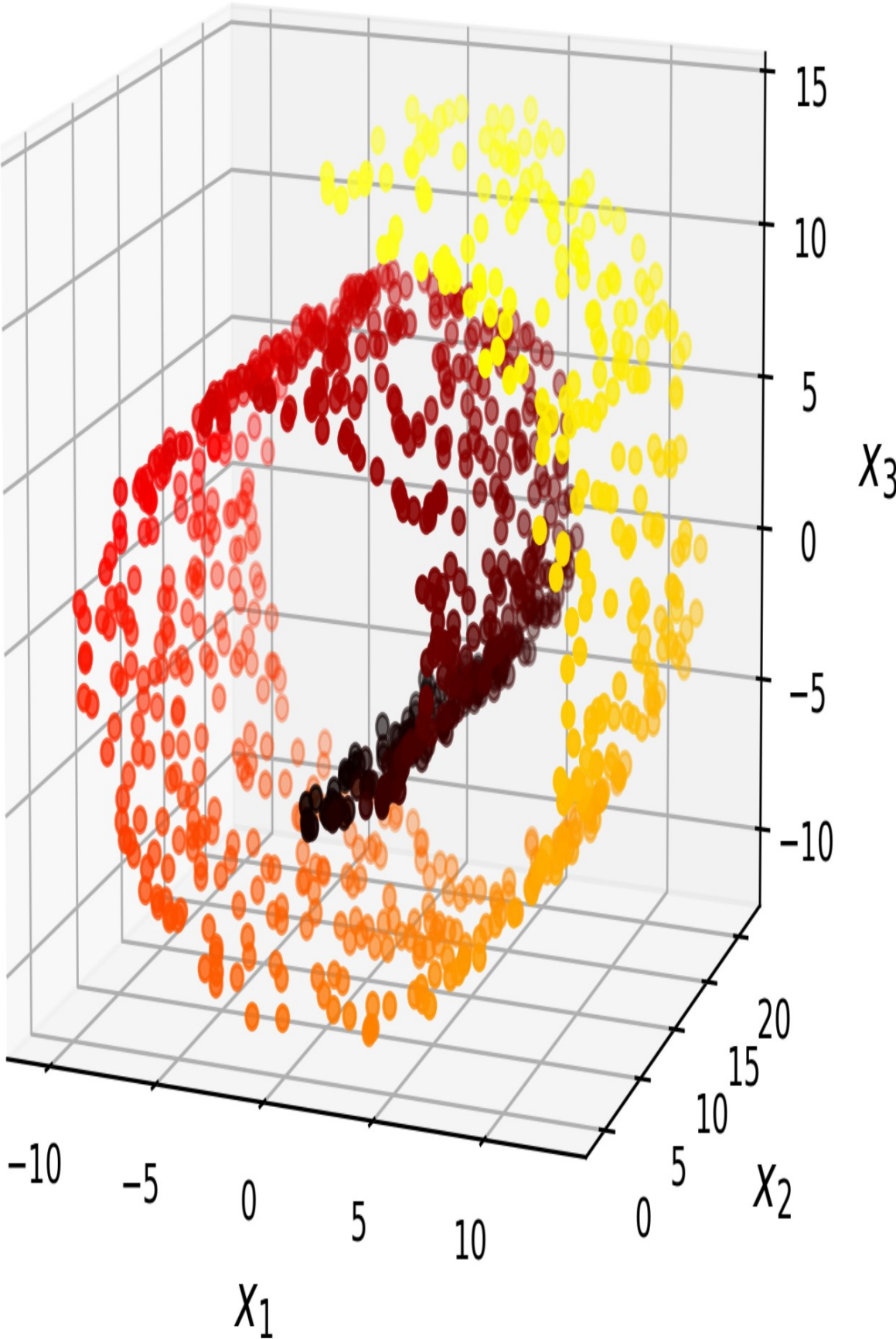
Hãy lưu ý rằng tất cả các trường hợp huấn luyện đều nằm gần một mặt phẳng: đây là một không gian con có chiều thấp hơn (2D) của không gian có chiều cao hơn (3D). Nếu ta chiếu mỗi trường hợp huấn luyện vuông góc lên không gian con này (như được biểu diễn bằng các đường chấm ngắn nối các trường hợp với mặt phẳng), ta sẽ thu được tập dữ liệu 2D mới được hiển thị trong **Hình 8-3**. Vậy là xong! Chúng ta vừa giảm chiều của tập dữ liệu từ 3D xuống 2D. Lưu ý rằng các trục tương ứng với các đặc trưng mới z và z : chúng là tọa độ của các hình chiếu trên mặt phẳng.



Hình 8-3. Tập dữ liệu 2D mới sau khi chiếu.

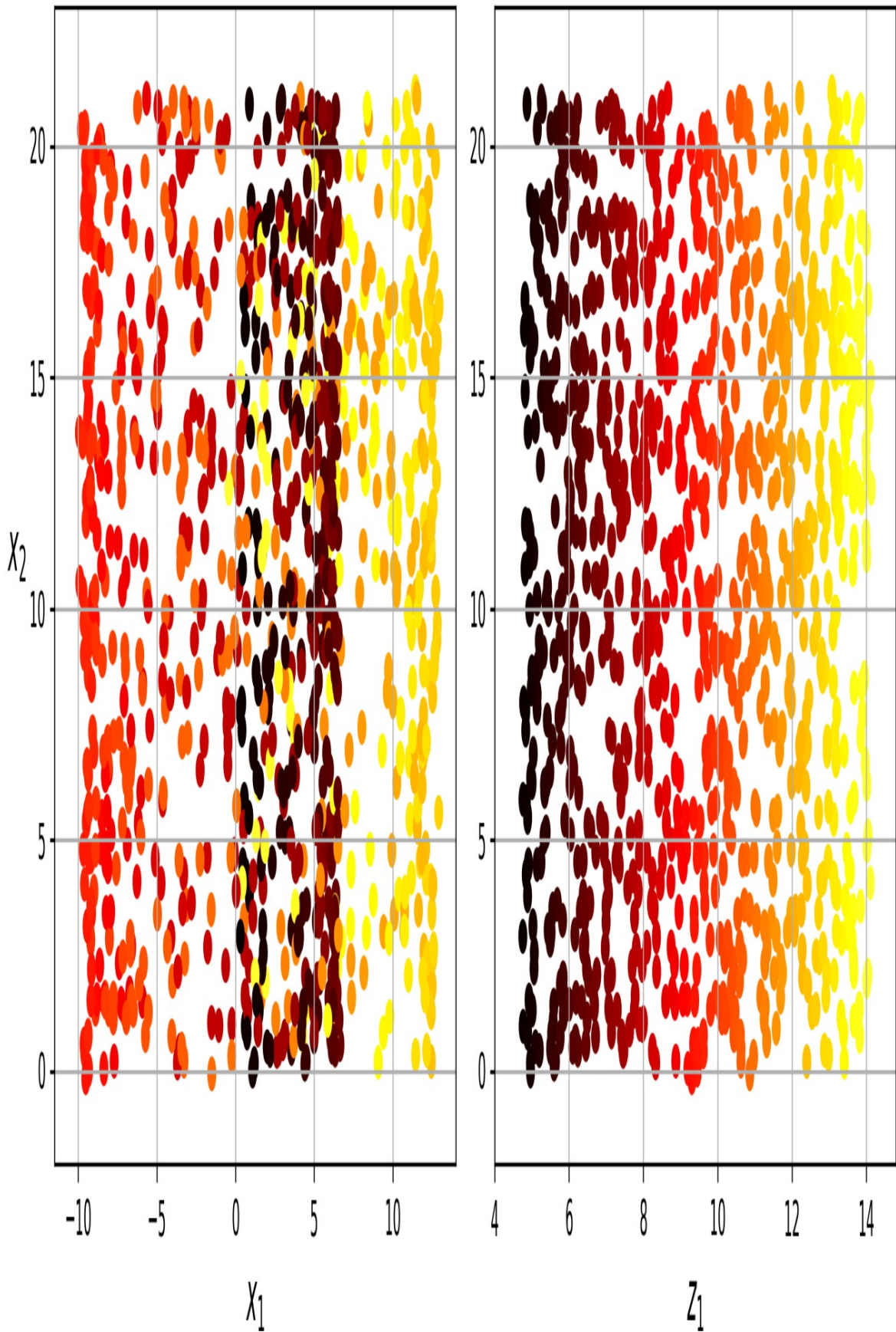
Học tập đa dạng

Tuy nhiên, phép chiếu không phải lúc nào cũng là phương pháp tốt nhất để giảm chiều dữ liệu. Trong nhiều trường hợp, không gian con có thể bị xoắn và uốn lượn, chẳng hạn như trong bộ dữ liệu đồ chơi cuộn Thụy Sĩ nổi tiếng được thể hiện trong **Hình 8-4**.



Hình 8-4. Bộ dữ liệu cuộn Thụy Sĩ

Đơn giản chỉ cần chiếu lên một mặt phẳng (ví dụ, bằng cách $3\times$) sẽ ép các lớp khác nhau của bánh cuộn Thụy Sĩ lại với nhau, như thể hiện ở phía bên trái của Hình 8-5. Có lẽ điều bạn muốn làm là trải cuộn giấy Thụy Sĩ ra để thu được tập dữ liệu 2D ở phía bên phải của Hình 8-5.



Hình 8-5. Nén bằng cách chiếu lên mặt phẳng (trái) so với việc trải phẳng cuộn bánh mì Thụy Sĩ (phải)

Bánh mì cuộn Thụy Sĩ là một ví dụ về đa tạp 2D. Nói một cách đơn giản, đa tạp 2D là một hình dạng 2D có thể uốn cong và xoắn trong không gian đa chiều.

Nói một cách tổng quát hơn, một đa tạp d chiều là một phần của không gian n chiều (trong đó $d < n$) mà về mặt cục bộ giống với một siêu phẳng d chiều. Trong trường hợp bánh mì cuộn Thụy Sĩ, $d = 2$ và $n = 3$: nó về mặt cục bộ giống với một mặt phẳng 2D, nhưng nó được cuộn trong không gian ba chiều.

Nhiều thuật toán giảm chiều hoạt động bằng cách mô hình hóa không gian đa tạp mà trên đó các dữ liệu huấn luyện nằm; điều này được gọi là Học trên Không gian Đa tạp (Manifold Learning). Nó dựa trên giả định về không gian đa tạp, còn được gọi là giả thuyết về không gian đa tạp, cho rằng hầu hết các tập dữ liệu đa chiều trong thế giới thực đều nằm gần một không gian đa tạp có chiều thấp hơn nhiều. Giả định này thường được quan sát thấy trong thực nghiệm.

Một lần nữa, hãy nghĩ về tập dữ liệu MNIST: tất cả các hình ảnh chữ số viết tay đều có một số điểm tương đồng. Chúng được tạo thành từ các đường nối liền nhau, viền màu trắng và ít nhiều được căn giữa. Nếu bạn tạo ngẫu nhiên các hình ảnh, chỉ một phần rất nhỏ trong số đó sẽ trông giống như chữ số viết tay. Nói cách khác, mức độ tự do mà bạn có được nếu cố gắng tạo ra một hình ảnh chữ số thấp hơn đáng kể so với mức độ tự do mà bạn có được nếu được phép tạo ra bất kỳ hình ảnh nào bạn muốn.

Những ràng buộc này có xu hướng thu hẹp tập dữ liệu vào một không gian đa chiều thấp hơn.

Giả định về đa tạp thường đi kèm với một giả định ngầm khác: rằng nhiệm vụ đang thực hiện (ví dụ: phân loại hoặc hồi quy) sẽ đơn giản hơn nếu được biểu diễn trong không gian đa tạp có chiều thấp hơn. Ví dụ, ở hàng trên cùng của Hình 8-6, cuộn bánh mì Thụy Sĩ được chia thành hai lớp: trong không gian 3D (bên trái), ranh giới quyết định sẽ khá phức tạp, nhưng trong không gian đa tạp 2D được trải phẳng (bên phải), ranh giới quyết định là một đường thẳng.

Tuy nhiên, giả định ngầm này không phải lúc nào cũng đúng. Ví dụ, ở hàng dưới cùng của Hình 8-6, ranh giới quyết định nằm ở $x = 5$.

Ranh giới quyết định này trông rất đơn giản trong không gian 3D ban đầu (một